My Project

Generated by Doxygen 1.9.8

1 ADS1115 Pin connection list: 1
1.1 Piezo pin connection . . . 1

2 Namespace Index 3
2.1 Namespace List . . . 3

3 Hierarchical Index 5
3.1 Class Hierarchy . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 File Index 9
5.1 File List . . . 9

6 Namespace Documentation 11
6.1 icm20948 Namespace Reference . . . 11
6.1.1 Typedef Documentation . . . 12
6.1.1.1 accel_settings . . . 12
6.1.1.2 gyro_settings . . . 12
6.1.1.3 magn_settings . . . 12
6.1.1.4 settings . . . 12
6.1.2 Enumeration Type Documentation . . . 12
6.1.2.1 accel_dlpf_config . . . 12
6.1.2.2 accel_scale . . . 13
6.1.2.3 gyro_dlpf_config . . . 13
6.1.2.4 gyro_scale . . . 13
6.1.2.5 magn_mode . . . 13
6.1.3 Function Documentation . . . 14
6.1.3.1 accel_dlpf_config_to_str() . . . 14
6.1.3.2 accel_scale_factor() . . . 14
6.1.3.3 accel_scale_to_str() . . . 14
6.1.3.4 gyro_dlpf_config_to_str() . . . 14
6.1.3.5 gyro_scale_factor() . . . 14
6.1.3.6 gyro_scale_to_str() . . . 14
6.1.3.7 magn_mode_to_str() . . . 14

7 Class Documentation 15
7.1 icm20948::accel_settings Struct Reference . . . 15
7.1.1 Constructor & Destructor Documentation . . . 15
7.1.1.1 accel_settings() . . . 15
7.1.2 Member Data Documentation . . . 15
7.1.2.1 dlpf_config . . . 15
7.1.2.2 dlpf_enable . . . 16

7.1.2.3 sample_rate_div . . . 16
7.1.2.4 scale . . . 16
7.2 ADS1115rpi Class Reference . . . 16
7.2.1 Member Typedef Documentation . . . 17
7.2.1.1 ADSCallbackInterface . . . 17
7.2.2 Constructor & Destructor Documentation . . . 17
7.2.2.1 ∼ADS1115rpi() . . . 17
7.2.3 Member Function Documentation . . . 17
7.2.3.1 dataReady() . . . 17
7.2.3.2 fullScaleVoltage() . . . 17
7.2.3.3 getADS1115settings() . . . 17
7.2.3.4 i2c_readConversion() . . . 18
7.2.3.5 i2c_readWord() . . . 18
7.2.3.6 i2c_selectDevice() . . . 18
7.2.3.7 i2c_writeWord() . . . 18
7.2.3.8 readOnce() . . . 18
7.2.3.9 registerCallback() . . . 18
7.2.3.10 setChannel() . . . 18
7.2.3.11 setRetryEnabled() . . . 18
7.2.3.12 start() . . . 18
7.2.3.13 stop() . . . 19
7.2.3.14 worker() . . . 19
7.2.4 Member Data Documentation . . . 19
7.2.4.1 ads1115settings . . . 19
7.2.4.2 adsCallbackInterface . . . 19
7.2.4.3 chip . . . 19
7.2.4.4 fd_i2c . . . 19
7.2.4.5 ISR_TIMEOUT_MS . . . 19
7.2.4.6 reg_config . . . 19
7.2.4.7 reg_hi_thres . . . 19
7.2.4.8 reg_lo_thres . . . 19
7.2.4.9 request . . . 20
7.2.4.10 retry_enabled_ . . . 20
7.2.4.11 running . . . 20
7.2.4.12 thr . . . 20
7.3 ADS1115settings Struct Reference . . . 20
7.3.1 Member Enumeration Documentation . . . 21
7.3.1.1 Input . . . 21
7.3.1.2 PGA . . . 21
7.3.1.3 SamplingRates . . . 21
7.3.2 Member Function Documentation . . . 22
7.3.2.1 getSamplingRate() . . . 22

7.3.3 Member Data Documentation . . . 22
7.3.3.1 address . . . 22
7.3.3.2 channel . . . 22
7.3.3.3 DEFAULT_ADS1115_ADDRESS . . . 22
7.3.3.4 DEFAULT_ALERT_RDY_TO_GPIO . . . 22
7.3.3.5 drdy_chip . . . 22
7.3.3.6 drdy_gpio . . . 22
7.3.3.7 i2c_bus . . . 22
7.3.3.8 pgaGain . . . 22
7.3.3.9 samplingRate . . . 23
7.4 PositionDetector::Config Struct Reference . . . 23
7.4.1 Member Data Documentation . . . 23
7.4.1.1 headingToleranceDeg . . . 23
7.4.1.2 maxG . . . 23
7.4.1.3 minDot . . . 23
7.4.1.4 minG . . . 23
7.4.1.5 refHeadingDeg . . . 24
7.4.1.6 refX . . . 24
7.4.1.7 refY . . . 24
7.4.1.8 refZ . . . 24
7.4.1.9 stabilitySamples . . . 24
7.5 icm20948::gyro_settings Struct Reference . . . 24
7.5.1 Constructor & Destructor Documentation . . . 25
7.5.1.1 gyro_settings() . . . 25
7.5.2 Member Data Documentation . . . 25
7.5.2.1 dlpf_config . . . 25
7.5.2.2 dlpf_enable . . . 25
7.5.2.3 sample_rate_div . . . 25
7.5.2.4 scale . . . 25
7.6 icm20948::ICM20948_I2C Class Reference . . . 25
7.6.1 Constructor & Destructor Documentation . . . 26
7.6.1.1 ICM20948_I2C() . . . 26
7.6.2 Member Function Documentation . . . 27
7.6.2.1 _chip_i2c_master_reset() . . . 27
7.6.2.2 _enable_data_ready_interrupt() . . . 27
7.6.2.3 _magnetometer_configured() . . . 27
7.6.2.4 _magnetometer_enable() . . . 27
7.6.2.5 _magnetometer_init() . . . 27
7.6.2.6 _magnetometer_set_mode() . . . 27
7.6.2.7 _magnetometer_set_readout() . . . 27
7.6.2.8 _read_bit() . . . 27
7.6.2.9 _read_block_bytes() . . . 27

7.6.2.10 _read_byte() . . . 28
7.6.2.11 _read_mag_byte() . . . 28
7.6.2.12 _set_accel_range_dlpf() . . . 28
7.6.2.13 _set_accel_sample_rate_div() . . . 28
7.6.2.14 _set_bank() . . . 28
7.6.2.15 _set_gyro_range_dlpf() . . . 28
7.6.2.16 _set_gyro_sample_rate_div() . . . 28
7.6.2.17 _write_bit() . . . 28
7.6.2.18 _write_byte() . . . 29
7.6.2.19 _write_mag_byte() . . . 29
7.6.2.20 check_DRDY_INT() . . . 29
7.6.2.21 init() . . . 29
7.6.2.22 read_accel_gyro() . . . 29
7.6.2.23 read_magn() . . . 29
7.6.2.24 read_sample() . . . 29
7.6.2.25 reset() . . . 29
7.6.2.26 set_settings() . . . 29
7.6.2.27 wake() . . . 30
7.6.3 Member Data Documentation . . . 30
7.6.3.1 _accel_scale_factor . . . 30
7.6.3.2 _current_bank . . . 30
7.6.3.3 _gyro_scale_factor . . . 30
7.6.3.4 _i2c_address . . . 30
7.6.3.5 _i2c_bus . . . 30
7.6.3.6 _i2c_fd . . . 30
7.6.3.7 _magn_scale_factor . . . 30
7.6.3.8 accel . . . 30
7.6.3.9 gyro . . . 30
7.6.3.10 magn . . . 31
7.6.3.11 settings . . . 31
7.7 IMUReader Class Reference . . . 31
7.7.1 Member Typedef Documentation . . . 32
7.7.1.1 SampleCallback . . . 32
7.7.2 Constructor & Destructor Documentation . . . 32
7.7.2.1 IMUReader() . . . 32
7.7.2.2 ~IMUReader() . . . 32
7.7.3 Member Function Documentation . . . 32
7.7.3.1 getIMU() . . . 32
7.7.3.2 init() . . . 32
7.7.3.3 setCallback() . . . 32
7.7.3.4 start() . . . 32
7.7.3.5 stop() . . . 32

7.7.3.6 worker() . . . 33
7.7.4 Member Data Documentation . . . 33
7.7.4.1 callback_ . . . 33
7.7.4.2 chip_ . . . 33
7.7.4.3 gpio_fd_ . . . 33
7.7.4.4 gpiochip_path_ . . . 33
7.7.4.5 imu_ . . . 33
7.7.4.6 rdy_line_ . . . 33
7.7.4.7 request_ . . . 33
7.7.4.8 running_ . . . 33
7.7.4.9 stop_fd_ . . . 33
7.7.4.10 worker_thread_ . . . 34
7.8 icm20948::IMUSample Struct Reference . . . 34
7.8.1 Member Data Documentation . . . 34
7.8.1.1 ax . . . 34
7.8.1.2 ay . . . 34
7.8.1.3 az . . . 34
7.8.1.4 gx . . . 34
7.8.1.5 gy . . . 35
7.8.1.6 gz . . . 35
7.8.1.7 mx . . . 35
7.8.1.8 my . . . 35
7.8.1.9 mz . . . 35
7.9 LEDController Class Reference . . . 35
7.9.1 Detailed Description . . . 36
7.9.2 Constructor & Destructor Documentation . . . 36
7.9.2.1 LEDController() . . . 36
7.9.2.2 ∼LEDController() . . . 37
7.9.3 Member Function Documentation . . . 37
7.9.3.1 allOff() . . . 37
7.9.3.2 flashGreen() . . . 37
7.9.3.3 greenOff() . . . 37
7.9.3.4 greenOn() . . . 37
7.9.3.5 service() . . . 37
7.9.3.6 set() . . . 38
7.9.3.7 updateOutputs() . . . 38
7.9.4 Member Data Documentation . . . 38
7.9.4.1 greenActive_ . . . 38
7.9.4.2 greenOffTime_ . . . 38
7.10 LEDControllerSettings Struct Reference . . . 38
7.10.1 Detailed Description . . . 39
7.10.2 Member Data Documentation . . . 39

7.10.2.1 chipNumber . . . 39
7.10.2.2 greenGpio . . . 39
7.11 icm20948::magn_settings Struct Reference . . . 39
7.11.1 Constructor & Destructor Documentation . . . 39
7.11.1.1 magn_settings() . . . 39
7.11.2 Member Data Documentation . . . 40
7.11.2.1 mode . . . 40
7.12 PiezoEventDetector Class Reference . . . 40
7.12.1 Member Typedef Documentation . . . 40
7.12.1.1 ForceCallback . . . 40
7.12.2 Constructor & Destructor Documentation . . . 41
7.12.2.1 PiezoEventDetector() . . . 41
7.12.3 Member Function Documentation . . . 41
7.12.3.1 processSample() . . . 41
7.12.3.2 reset() . . . 41
7.12.3.3 setForceCallback() . . . 41
7.12.4 Member Data Documentation . . . 41
7.12.4.1 filter_init_ . . . 41
7.12.4.2 filtered_v_ . . . 41
7.12.4.3 forceCallback_ . . . 41
7.12.4.4 led_ . . . 41
7.12.4.5 pressed_ . . . 42
7.12.4.6 settings_ . . . 42
7.13 PiezoEventDetectorSettings Struct Reference . . . 42
7.13.1 Member Data Documentation . . . 42
7.13.1.1 enableDebugPrints . . . 42
7.13.1.2 pressThreshold . . . 42
7.13.1.3 releaseThreshold . . . 42
7.14 PositionDetector Class Reference . . . 43
7.14.1 Member Typedef Documentation . . . 43
7.14.1.1 StateCallback . . . 43
7.14.2 Constructor & Destructor Documentation . . . 44
7.14.2.1 PositionDetector() [1/2] . . . 44
7.14.2.2 PositionDetector() [2/2] . . . 44
7.14.2.3 ~PositionDetector() . . . 44
7.14.3 Member Function Documentation . . . 44
7.14.3.1 onSample() . . . 44
7.14.3.2 resetState() . . . 44
7.14.3.3 setCallback() . . . 44
7.14.3.4 wrapAngleDeg() . . . 44
7.14.4 Member Data Documentation . . . 45
7.14.4.1 callback . . . 45

7.14.4.2 cfg . . . 45
7.14.4.3 confirmed . . . 45
7.14.4.4 nRefX . . . 45
7.14.4.5 nRefY . . . 45
7.14.4.6 nRefZ . . . 45
7.14.4.7 stableCount . . . 45
7.15 RPI_PWM Class Reference . . . 45
7.15.1 Constructor & Destructor Documentation . . . 46
7.15.1.1 RPI_PWM() . . . 46
7.15.1.2 ~RPI_PWM() . . . 46
7.15.2 Member Function Documentation . . . 46
7.15.2.1 getDutyCycle() . . . 46
7.15.2.2 loop() . . . 46
7.15.2.3 setDutyCycle() . . . 47
7.15.2.4 start() . . . 47
7.15.2.5 stop() . . . 47
7.15.2.6 usToTimerspec() . . . 47
7.15.2.7 waitUs() . . . 47
7.15.3 Member Data Documentation . . . 47
7.15.3.1 chip_ . . . 47
7.15.3.2 duty_ . . . 47
7.15.3.3 request_ . . . 47
7.15.3.4 running_ . . . 47
7.15.3.5 thread_ . . . 48
7.16 icm20948::settings Struct Reference . . . 48
7.16.1 Constructor & Destructor Documentation . . . 48
7.16.1.1 settings() [1/2] . . . 48
7.16.1.2 settings() [2/2] . . . 48
7.16.2 Member Data Documentation . . . 48
7.16.2.1 accel . . . 48
7.16.2.2 gyro . . . 49
7.16.2.3 magn . . . 49
7.17 SimpleLEDController Class Reference . . . 49
7.17.1 Detailed Description . . . 50
7.17.2 Constructor & Destructor Documentation . . . 51
7.17.2.1 SimpleLEDController() [1/2] . . . 51
7.17.2.2 ~SimpleLEDController() . . . 51
7.17.2.3 SimpleLEDController() [2/2] . . . 51
7.17.3 Member Function Documentation . . . 51
7.17.3.1 flashGreen() . . . 51
7.17.3.2 service() . . . 52
7.17.3.3 set() . . . 52

7.17.4 Member Data Documentation . . . 52
7.17.4.1 chip . . . 52
7.17.4.2 chipNumber . . . 52
7.17.4.3 pin . . . 52
7.17.4.4 request . . . 53
7.18 SwingCalibrator Class Reference . . . 53
7.18.1 Detailed Description . . . 54
7.18.2 Member Typedef Documentation . . . 54
7.18.2.1 DoneCallback . . . 54
7.18.3 Member Enumeration Documentation . . . 54
7.18.3.1 State . . . 54
7.18.4 Constructor & Destructor Documentation . . . 54
7.18.4.1 SwingCalibrator() . . . 54
7.18.5 Member Function Documentation . . . 54
7.18.5.1 accumulate() . . . 54
7.18.5.2 biasX() . . . 55
7.18.5.3 biasY() . . . 55
7.18.5.4 biasZ() . . . 55
7.18.5.5 feed() . . . 55
7.18.5.6 isReady() . . . 55
7.18.5.7 setCallback() . . . 55
7.18.5.8 triggerRecal() . . . 55
7.18.6 Member Data Documentation . . . 55
7.18.6.1 bias_x_ . . . 55
7.18.6.2 bias_y_ . . . 56
7.18.6.3 bias_z_ . . . 56
7.18.6.4 callback_ . . . 56
7.18.6.5 count_ . . . 56
7.18.6.6 initial_samples_ . . . 56
7.18.6.7 recal_samples_ . . . 56
7.18.6.8 state_ . . . 56
7.18.6.9 sum_x_ . . . 56
7.18.6.10 sum_y_ . . . 56
7.18.6.11 sum_z_ . . . 57
7.19 SwingDetector Class Reference . . . 57
7.19.1 Member Typedef Documentation . . . 57
7.19.1.1 EventCallback . . . 57
7.19.2 Member Enumeration Documentation . . . 57
7.19.2.1 Level . . . 57
7.19.3 Constructor & Destructor Documentation . . . 58
7.19.3.1 SwingDetector() . . . 58
7.19.4 Member Function Documentation . . . 58

7.19.4.1 classify() . . . 58
7.19.4.2 detect() . . . 58
7.19.4.3 label() . . . 58
7.19.4.4 reset() . . . 58
7.19.4.5 setCallback() . . . 58
7.19.5 Member Data Documentation . . . 58
7.19.5.1 callback_ . . . 58
7.19.5.2 last_level_ . . . 59
7.20 SwingFeedback Class Reference . . . 59
7.20.1 Member Typedef Documentation . . . 60
7.20.1.1 ResetCallback . . . 60
7.20.2 Constructor & Destructor Documentation . . . 60
7.20.2.1 SwingFeedback() . . . 60
7.20.3 Member Function Documentation . . . 60
7.20.3.1 checkTimeout() . . . 60
7.20.3.2 forceOff() . . . 60
7.20.3.3 onLevel() . . . 60
7.20.3.4 setResetCallback() . . . 60
7.20.3.5 worker() . . . 60
7.20.4 Member Data Documentation . . . 60
7.20.4.1 busy_ . . . 60
7.20.4.2 busy_since_ms_ . . . 61
7.20.4.3 pending_level_ . . . 61
7.20.4.4 pwm_ . . . 61
7.20.4.5 reset_cb_ . . . 61
7.20.4.6 TIMEOUT_MS . . . 61
7.21 SwingProcessor Class Reference . . . 61
7.21.1 Member Typedef Documentation . . . 62
7.21.1.1 MagnitudeCallback . . . 62
7.21.1.2 PositionStateCallback . . . 62
7.21.2 Constructor & Destructor Documentation . . . 63
7.21.2.1 SwingProcessor() . . . 63
7.21.3 Member Function Documentation . . . 63
7.21.3.1 isActive() . . . 63
7.21.3.2 onForceReady() . . . 63
7.21.3.3 onPositionReady() . . . 63
7.21.3.4 onSample() . . . 63
7.21.3.5 operator()() . . . 63
7.21.3.6 reset() . . . 64
7.21.3.7 setMagnitudeCallback() . . . 64
7.21.3.8 setPositionCallback() . . . 64
7.21.4 Member Data Documentation . . . 64

7.21.4.1 accel_deadband_ . . . 64
7.21.4.2 calibrator_ . . . 64
7.21.4.3 COOLDOWN_MS . . . 64
7.21.4.4 isActive_ . . . 64
7.21.4.5 isArmed_ . . . 64
7.21.4.6 mag_callback_ . . . 64
7.21.4.7 pos_callback_ . . . 64
7.21.4.8 reset_time_ . . . 64

8 File Documentation **65**

8.1 src/libs/IMU/Icm20948driver.cpp File Reference . . . 65
8.2 src/libs/IMU/imureader.cpp File Reference . . . 65
8.3 src/libs/IMU/Include/i2c_mutex.hpp File Reference . . . 65
8.3.1 Function Documentation . . . 66
8.3.1.1 i2c1_mutex() . . . 66
8.4 i2c_mutex.hpp . . . 66
8.5 src/libs/IMU/Include/Icm20948driver.hpp File Reference . . . 66
8.5.1 Macro Definition Documentation . . . 69
8.5.1.1 AK09916_CNTL2_ADDR . . . 69
8.5.1.2 ICM20948_ACCEL_CONFIG_1_ADDR . . . 69
8.5.1.3 ICM20948_ACCEL_CONFIG_1_BANK . . . 69
8.5.1.4 ICM20948_ACCEL_OUT_BANK . . . 69
8.5.1.5 ICM20948_ACCEL_SMPLRT_DIV_1_ADDR . . . 69
8.5.1.6 ICM20948_ACCEL_SMPLRT_DIV_1_BANK . . . 69
8.5.1.7 ICM20948_ACCEL_SMPLRT_DIV_2_ADDR . . . 69
8.5.1.8 ICM20948_ACCEL_SMPLRT_DIV_2_BANK . . . 69
8.5.1.9 ICM20948_ACCEL_XOUT_H_ADDR . . . 70
8.5.1.10 ICM20948_ACCEL_XOUT_L_ADDR . . . 70
8.5.1.11 ICM20948_ACCEL_YOUT_H_ADDR . . . 70
8.5.1.12 ICM20948_ACCEL_YOUT_L_ADDR . . . 70
8.5.1.13 ICM20948_ACCEL_ZOUT_H_ADDR . . . 70
8.5.1.14 ICM20948_ACCEL_ZOUT_L_ADDR . . . 70
8.5.1.15 ICM20948_BANK0_WHO_AM_I_VALUE . . . 70
8.5.1.16 ICM20948_EXT_SLV_SENS_DATA_00_ADDR . . . 70
8.5.1.17 ICM20948_EXT_SLV_SENS_DATA_00_BANK . . . 70
8.5.1.18 ICM20948_GYRO_CONFIG_1_ADDR . . . 70
8.5.1.19 ICM20948_GYRO_CONFIG_1_BANK . . . 71
8.5.1.20 ICM20948_GYRO_OUT_BANK . . . 71
8.5.1.21 ICM20948_GYRO_SMPLRT_DIV_ADDR . . . 71
8.5.1.22 ICM20948_GYRO_SMPLRT_DIV_BANK . . . 71
8.5.1.23 ICM20948_GYRO_XOUT_H_ADDR . . . 71
8.5.1.24 ICM20948_GYRO_XOUT_L_ADDR . . . 71

8.5.1.25 ICM20948_GYRO_YOUT_H_ADDR . . . 71
8.5.1.26 ICM20948_GYRO_YOUT_L_ADDR . . . 71
8.5.1.27 ICM20948_GYRO_ZOUT_H_ADDR . . . 71
8.5.1.28 ICM20948_GYRO_ZOUT_L_ADDR . . . 71
8.5.1.29 ICM20948_I2C_ADDR . . . 72
8.5.1.30 ICM20948_I2C_MST_CTRL_ADDR . . . 72
8.5.1.31 ICM20948_I2C_MST_CTRL_BANK . . . 72
8.5.1.32 ICM20948_I2C_MST_STATUS_ADDR . . . 72
8.5.1.33 ICM20948_I2C_MST_STATUS_BANK . . . 72
8.5.1.34 ICM20948_I2C_SLV0_ADDR_ADDR . . . 72
8.5.1.35 ICM20948_I2C_SLV0_ADDR_BANK . . . 72
8.5.1.36 ICM20948_I2C_SLV0_CTRL_ADDR . . . 72
8.5.1.37 ICM20948_I2C_SLV0_CTRL_BANK . . . 72
8.5.1.38 ICM20948_I2C_SLV0_REG_ADDR . . . 72
8.5.1.39 ICM20948_I2C_SLV0_REG_BANK . . . 73
8.5.1.40 ICM20948_I2C_SLV4_ADDR_ADDR . . . 73
8.5.1.41 ICM20948_I2C_SLV4_ADDR_BANK . . . 73
8.5.1.42 ICM20948_I2C_SLV4_CTRL_ADDR . . . 73
8.5.1.43 ICM20948_I2C_SLV4_CTRL_BANK . . . 73
8.5.1.44 ICM20948_I2C_SLV4_DI_ADDR . . . 73
8.5.1.45 ICM20948_I2C_SLV4_DI_BANK . . . 73
8.5.1.46 ICM20948_I2C_SLV4_DO_ADDR . . . 73
8.5.1.47 ICM20948_I2C_SLV4_DO_BANK . . . 73
8.5.1.48 ICM20948_I2C_SLV4_REG_ADDR . . . 73
8.5.1.49 ICM20948_I2C_SLV4_REG_BANK . . . 74
8.5.1.50 ICM20948_INT_ENABLE_1_ADDR . . . 74
8.5.1.51 ICM20948_INT_ENABLE_1_BANK . . . 74
8.5.1.52 ICM20948_INT_PIN_CFG_ADDR . . . 74
8.5.1.53 ICM20948_INT_PIN_CFG_BANK . . . 74
8.5.1.54 ICM20948_INT_STATUS_1_ADDR . . . 74
8.5.1.55 ICM20948_INT_STATUS_1_BANK . . . 74
8.5.1.56 ICM20948_INT_STATUS_ADDR . . . 74
8.5.1.57 ICM20948_INT_STATUS_BANK . . . 74
8.5.1.58 ICM20948_MAGN_I2C_ADDR . . . 74
8.5.1.59 ICM20948_PWR_MGMT_1_ADDR . . . 75
8.5.1.60 ICM20948_PWR_MGMT_1_BANK . . . 75
8.5.1.61 ICM20948_REG_BANK_SEL_ADDR . . . 75
8.5.1.62 ICM20948_REG_BANK_SEL_BANK0_VALUE . . . 75
8.5.1.63 ICM20948_REG_BANK_SEL_BANK1_VALUE . . . 75
8.5.1.64 ICM20948_REG_BANK_SEL_BANK2_VALUE . . . 75
8.5.1.65 ICM20948_REG_BANK_SEL_BANK3_VALUE . . . 75
8.5.1.66 ICM20948_TEMP_OUT_H_ADDR . . . 75

8.5.1.67 ICM20948_TEMP_OUT_L_ADDR . . . 75
8.5.1.68 ICM20948_USER_CTRL_ADDR . . . 75
8.5.1.69 ICM20948_USER_CTRL_BANK . . . 76
8.5.1.70 ICM20948_WHO_AM_I_ADDR . . . 76
8.5.1.71 ICM20948_WHO_AM_I_BANK . . . 76
8.6 Icm20948driver.hpp . . . 76
8.7 src/libs/IMU/Include/imureader.hpp File Reference . . . 80
8.8 imureader.hpp . . . 80
8.9 src/libs/IMU_math/Include/positiondetector.hpp File Reference . . . 82
8.10 positiondetector.hpp . . . 82
8.11 src/libs/IMU_math/Include/SwingCalibrator.hpp File Reference . . . 83
8.12 SwingCalibrator.hpp . . . 83
8.13 src/libs/IMU_math/Include/SwingDetector.hpp File Reference . . . 84
8.14 SwingDetector.hpp . . . 84
8.15 src/libs/IMU_math/Include/SwingProcessor.hpp File Reference . . . 84
8.15.1 Variable Documentation . . . 85
8.15.1.1 ACCEL_DEADBAND . . . 85
8.15.1.2 CALIB_SAMPLES . . . 85
8.15.1.3 I2C_BUS . . . 85
8.15.1.4 RECAL_SAMPLES . . . 85
8.16 SwingProcessor.hpp . . . 85
8.17 src/libs/IMU_math/positiondetector.cpp File Reference . . . 86
8.18 src/libs/IMU_math/SwingCalibrator.cpp File Reference . . . 86
8.19 src/libs/IMU_math/SwingDetector.cpp File Reference . . . 86
8.20 src/libs/IMU_math/SwingProcessor.cpp File Reference . . . 87
8.21 src/libs/LEDs/Include/led_controller.h File Reference . . . 87
8.22 led_controller.h . . . 87
8.23 src/libs/LEDs/Include/ledcallback.hpp File Reference . . . 88
8.24 ledcallback.hpp . . . 88
8.25 src/libs/LEDs/led_controller.cpp File Reference . . . 88
8.26 src/libs/LEDs/ledcallback.cpp File Reference . . . 89
8.27 src/libs/Motor/Include/rpi_pwm.h File Reference . . . 89
8.27.1 Variable Documentation . . . 89
8.27.1.1 PWM_FREQ_HZ . . . 89
8.27.1.2 PWM_GPIO_LINE . . . 89
8.28 rpi_pwm.h . . . 90
8.29 src/libs/Motor/Include/swingfeedback.h File Reference . . . 91
8.30 swingfeedback.h . . . 91
8.31 src/libs/Motor/swingfeedback.cpp File Reference . . . 92
8.31.1 Function Documentation . . . 92
8.31.1.1 nowMs() . . . 92
8.32 src/libs/Piezo/ads1115rpi.cpp File Reference . . . 93

8.32.1 Function Documentation 93
8.32.1.1 makeDefaultADS1115Settings() 93
8.33 src/libs/Piezo/event_detector.cpp File Reference 93
8.34 src/libs/Piezo/Include/ads1115rpi.h File Reference 93
8.34.1 Macro Definition Documentation 94
8.34.1.1 DEBUG 94
8.34.2 Function Documentation 94
8.34.2.1 makeDefaultADS1115Settings() 94
8.35 ads1115rpi.h 94
8.36 src/libs/Piezo/Include/event_detector.h File Reference 96
8.37 event_detector.h 96
8.38 src/libs/Piezo/ReadMe.md File Reference 97
8.39 src/main.cpp File Reference 97
8.39.1 Function Documentation 98
8.39.1.1 handle_signal() 98
8.39.1.2 main() 98
8.39.1.3 running() 98

Index **99**

Chapter 1

ADS1115 Pin connection list:

```
* VDD - Pin 1
* SDA - Pin 3
* SCL - Pin 5
* A0 - Pin 11
* ADDR - GND
* GND - GND
* ALRT - positive piezo
```

1.1 Piezo pin connection

- Negative - GND
- Positive - ALRT
- Both - connect to resistor

Connections to the rasbperry pi

breadboard wire connections (birds eye view)

breadboard wire connections (side view)

breadborad wire connection (3rd view)

ADS1115 Wire connections

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

icm20948 11

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

icm20948::accel_settings 15
ADS1115rpi 16
ADS1115settings 20
PositionDetector::Config 23
icm20948::gyro_settings 24
icm20948::ICM20948_I2C 25
IMUReader 31
icm20948::IMUSample 34
LEDControllerSettings 38
icm20948::magn_settings 39
PiezoEventDetector 40
PiezoEventDetectorSettings 42
PositionDetector 43
    SwingProcessor 61
RPI_PWM 45
icm20948::settings 48
SimpleLEDController 49
    LEDController 35
SwingCalibrator 53
SwingDetector 57
SwingFeedback 59

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

icm20948::accel_settings 15
ADS1115rpi 16
ADS1115settings 20
PositionDetector::Config 23
icm20948::gyro_settings 24
icm20948::ICM20948_I2C 25
IMUReader 31
icm20948::IMUSample 34
LEDController
Declares the LED controller class and its main functions for setting, switching, flashing, and updating the LED output 35
LEDControllerSettings
This header declares the LED controller settings and class 38
icm20948::magn_settings 39
PiezoEventDetector 40
PiezoEventDetectorSettings 42
PositionDetector 43
RPI_PWM 45
icm20948::settings 48
SimpleLEDController
This header declares a simple LED controller base class. This is to control the LED associated with the IMU swing position 49
SwingCalibrator 53
SwingDetector 57
SwingFeedback 59
SwingProcessor 61

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

src/main.cpp . . . 97
src/libs/IMU/Icm20948driver.cpp . . . 65
src/libs/IMU/imureader.cpp . . . 65
src/libs/IMU/Include/i2c_mutex.hpp . . . 65
src/libs/IMU/Include/Icm20948driver.hpp . . . 66
src/libs/IMU/Include/imureader.hpp . . . 80
src/libs/IMU_math/positiondetector.cpp . . . 86
src/libs/IMU_math/SwingCalibrator.cpp . . . 86
src/libs/IMU_math/SwingDetector.cpp . . . 86
src/libs/IMU_math/SwingProcessor.cpp . . . 87
src/libs/IMU_math/Include/positiondetector.hpp . . . 82
src/libs/IMU_math/Include/SwingCalibrator.hpp . . . 83
src/libs/IMU_math/Include/SwingDetector.hpp . . . 84
src/libs/IMU_math/Include/SwingProcessor.hpp . . . 84
src/libs/LEDs/led_controller.cpp . . . 88
src/libs/LEDs/ledcallback.cpp . . . 89
src/libs/LEDs/Include/led_controller.h . . . 87
src/libs/LEDs/Include/ledcallback.hpp . . . 88
src/libs/Motor/swingfeedback.cpp . . . 92
src/libs/Motor/Include/rpi_pwm.h . . . 89
src/libs/Motor/Include/swingfeedback.h . . . 91
src/libs/Piezo/ads1115rpi.cpp . . . 93
src/libs/Piezo/event_detector.cpp . . . 93
src/libs/Piezo/Include/ads1115rpi.h . . . 93
src/libs/Piezo/Include/event_detector.h . . . 96

Chapter 6

Namespace Documentation

6.1 icm20948 Namespace Reference

Classes

- struct accel_settings
- struct gyro_settings
- class ICM20948_I2C
- struct IMUSample
- struct magn_settings
- struct settings

Typedefs

- typedef struct icm20948::accel_settings accel_settings
- typedef struct icm20948::gyro_settings gyro_settings
- typedef struct icm20948::magn_settings magn_settings
- typedef struct icm20948::settings settings

Enumerations

- enum accel_scale { ACCEL_2G = 0 , ACCEL_4G , ACCEL_8G , ACCEL_16G }
- enum accel_dlpf_config {
  ACCEL_DLPF_246HZ = 0 , ACCEL_DLPF_246HZ_2 , ACCEL_DLPF_111_4HZ , ACCEL_DLPF_50_4HZ ,
  ACCEL_DLPF_23_9HZ , ACCEL_DLPF_11_5HZ , ACCEL_DLPF_5_7HZ , ACCEL_DLPF_473HZ }
- enum gyro_scale { GYRO_250DPS = 0 , GYRO_500DPS , GYRO_1000DPS , GYRO_2000DPS }
- enum gyro_dlpf_config {
  GYRO_DLPF_196_6HZ = 0 , GYRO_DLPF_151_8HZ , GYRO_DLPF_119_5HZ , GYRO_DLPF_51_2HZ ,
  GYRO_DLPF_23_9HZ , GYRO_DLPF_11_6HZ , GYRO_DLPF_5_7HZ , GYRO_DLPF_361_4HZ }
- enum magn_mode {
  MAGN_SHUTDOWN = 0 , MAGN_SINGLE = 1 , MAGN_10HZ = 2 , MAGN_20HZ = 4 ,
  MAGN_50HZ = 6 , MAGN_100HZ = 8 , MAGN_SELF_TEST = 16 }

Functions

- float accel_scale_factor (accel_scale scale)
- std::string accel_scale_to_str (accel_scale scale)
- std::string accel_dlpf_config_to_str (accel_dlpf_config config)
- float gyro_scale_factor (gyro_scale scale)
- std::string gyro_scale_to_str (gyro_scale scale)
- std::string gyro_dlpf_config_to_str (gyro_dlpf_config config)
- std::string magn_mode_to_str (magn_mode mode)

6.1.1 Typedef Documentation

6.1.1.1 accel_settings

```
typedef struct icm20948::accel_settings icm20948::accel_settings
```

6.1.1.2 gyro_settings

```
typedef struct icm20948::gyro_settings icm20948::gyro_settings
```

6.1.1.3 magn_settings

```
typedef struct icm20948::magn_settings icm20948::magn_settings
```

6.1.1.4 settings

```
typedef struct icm20948::settings icm20948::settings
```

6.1.2 Enumeration Type Documentation

6.1.2.1 accel_dlpf_config

```
enum icm20948::accel_dlpf_config
```

Enumerator

ACCEL_DLPF_246HZ	
ACCEL_DLPF_246HZ↩ _2	
ACCEL_DLPF_111_4HZ	
ACCEL_DLPF_50_4HZ	
ACCEL_DLPF_23_9HZ	
ACCEL_DLPF_11_5HZ	
ACCEL_DLPF_5_7HZ	
ACCEL_DLPF_473HZ	

6.1.2.2 accel_scale

`enum icm20948::accel_scale`

Enumerator

ACCEL_2G	
ACCEL_4G	
ACCEL_8G	
ACCEL_16G	

6.1.2.3 gyro_dlpf_config

`enum icm20948::gyro_dlpf_config`

Enumerator

GYRO_DLPF_196_6HZ	
GYRO_DLPF_151_8HZ	
GYRO_DLPF_119_5HZ	
GYRO_DLPF_51_2HZ	
GYRO_DLPF_23_9HZ	
GYRO_DLPF_11_6HZ	
GYRO_DLPF_5_7HZ	
GYRO_DLPF_361_4HZ	

6.1.2.4 gyro_scale

`enum icm20948::gyro_scale`

Enumerator

GYRO_250DPS	
GYRO_500DPS	
GYRO_1000DPS	
GYRO_2000DPS	

6.1.2.5 magn_mode

`enum icm20948::magn_mode`

Enumerator

MAGN_SHUTDOWN	
MAGN_SINGLE	
MAGN_10HZ	

Enumerator

MAGN_20HZ	
MAGN_50HZ	
MAGN_100HZ	
MAGN_SELF_TEST	

6.1.3 Function Documentation

6.1.3.1 accel_dlpf_config_to_str()

```
std::string icm20948::accel_dlpf_config_to_str (
            accel_dlpf_config config )
```

6.1.3.2 accel_scale_factor()

```
float icm20948::accel_scale_factor (
            accel_scale scale )
```

6.1.3.3 accel_scale_to_str()

```
std::string icm20948::accel_scale_to_str (
            accel_scale scale )
```

6.1.3.4 gyro_dlpf_config_to_str()

```
std::string icm20948::gyro_dlpf_config_to_str (
            gyro_dlpf_config config )
```

6.1.3.5 gyro_scale_factor()

```
float icm20948::gyro_scale_factor (
            gyro_scale scale )
```

6.1.3.6 gyro_scale_to_str()

```
std::string icm20948::gyro_scale_to_str (
            gyro_scale scale )
```

6.1.3.7 magn_mode_to_str()

```
std::string icm20948::magn_mode_to_str (
            magn_mode mode )
```

Chapter 7

Class Documentation

7.1 icm20948::accel_settings Struct Reference

```
#include <Icm20948driver.hpp>
```

Public Member Functions

- accel_settings (uint16_t sample_rate_div=0, accel_scale scale=ACCEL_2G, bool dlpf_enable=true, accel_dlpf_config dlpf_config=ACCEL_DLPF_246HZ)

Public Attributes

- uint16_t sample_rate_div
- accel_scale scale
- bool dlpf_enable
- accel_dlpf_config dlpf_config

7.1.1 Constructor & Destructor Documentation

7.1.1.1 accel_settings()

```
icm20948::accel_settings::accel_settings (
            uint16_t sample_rate_div = 0,
            accel_scale scale = ACCEL_2G,
            bool dlpf_enable = true,
            accel_dlpf_config dlpf_config = ACCEL_DLPF_246HZ )  [inline]
```

7.1.2 Member Data Documentation

7.1.2.1 dlpf_config

```
accel_dlpf_config icm20948::accel_settings::dlpf_config
```

7.1.2.2 dlpf_enable

```
bool icm20948::accel_settings::dlpf_enable
```

7.1.2.3 sample_rate_div

```
uint16_t icm20948::accel_settings::sample_rate_div
```

7.1.2.4 scale

```
accel_scale icm20948::accel_settings::scale
```

The documentation for this struct was generated from the following file:

- src/libs/IMU/Include/Icm20948driver.hpp

7.2 ADS1115rpi Class Reference

```
#include <ads1115rpi.h>
```

Collaboration diagram for ADS1115rpi:

Public Types

- using ADSCallbackInterface = std::function< void(float)>

Public Member Functions

- ~ADS1115rpi ()
- void registerCallback (ADSCallbackInterface ci)
- void setChannel (ADS1115settings::Input channel)
- void start (ADS1115settings settings=ADS1115settings(), bool spawn_worker=true)
- ADS1115settings getADS1115settings () const
- void stop ()
- int readOnce ()
- float fullScaleVoltage () const
- void setRetryEnabled (bool enabled)

Private Member Functions

- void dataReady ()
- void worker ()
- void i2c_selectDevice ()
- void i2c_writeWord (uint8_t reg, unsigned data)
- unsigned i2c_readWord (uint8_t reg)
- int i2c_readConversion ()

Private Attributes

- ADS1115settings ads1115settings
- const uint8_t reg_config = 1
- const uint8_t reg_lo_thres = 2
- const uint8_t reg_hi_thres = 3
- std::shared_ptr< gpiod::chip > chip
- std::shared_ptr< gpiod::line_request > request
- std::thread thr
- int fd_i2c = -1
- bool running = false
- ADSCallbackInterface adsCallbackInterface
- std::atomic< bool > retry_enabled_ {false}

Static Private Attributes

- static constexpr int64_t ISR_TIMEOUT_MS = 500

7.2.1 Member Typedef Documentation

7.2.1.1 ADSCallbackInterface

```
using ADS1115rpi::ADSCallbackInterface = std::function<void(float)>
```

7.2.2 Constructor & Destructor Documentation

7.2.2.1 ~ADS1115rpi()

```
ADS1115rpi::~ADS1115rpi ( )  [inline]
```

7.2.3 Member Function Documentation

7.2.3.1 dataReady()

```
void ADS1115rpi::dataReady ( )  [private]
```

7.2.3.2 fullScaleVoltage()

```
float ADS1115rpi::fullScaleVoltage ( ) const  [inline]
```

7.2.3.3 getADS1115settings()

```
ADS1115settings ADS1115rpi::getADS1115settings ( ) const  [inline]
```

7.2.3.4 i2c_readConversion()

```
int ADS1115rpi::i2c_readConversion ( ) [private]
```

7.2.3.5 i2c_readWord()

```
unsigned ADS1115rpi::i2c_readWord (
            uint8_t reg ) [private]
```

7.2.3.6 i2c_selectDevice()

```
void ADS1115rpi::i2c_selectDevice ( ) [private]
```

7.2.3.7 i2c_writeWord()

```
void ADS1115rpi::i2c_writeWord (
            uint8_t reg,
            unsigned data ) [private]
```

7.2.3.8 readOnce()

```
int ADS1115rpi::readOnce ( )
```

7.2.3.9 registerCallback()

```
void ADS1115rpi::registerCallback (
            ADSCallbackInterface ci ) [inline]
```

7.2.3.10 setChannel()

```
void ADS1115rpi::setChannel (
            ADS1115settings::Input channel )
```

7.2.3.11 setRetryEnabled()

```
void ADS1115rpi::setRetryEnabled (
            bool enabled ) [inline]
```

7.2.3.12 start()

```
void ADS1115rpi::start (
            ADS1115settings settings = ADS1115settings(),
            bool spawn_worker = true )
```

7.2.3.13 stop()

```
void ADS1115rpi::stop ( )
```

7.2.3.14 worker()

```
void ADS1115rpi::worker ( ) [private]
```

7.2.4 Member Data Documentation

7.2.4.1 ads1115settings

```
ADS1115settings ADS1115rpi::ads1115settings [private]
```

7.2.4.2 adsCallbackInterface

```
ADSCallbackInterface ADS1115rpi::adsCallbackInterface [private]
```

7.2.4.3 chip

```
std::shared_ptr<gpiod::chip> ADS1115rpi::chip [private]
```

7.2.4.4 fd_i2c

```
int ADS1115rpi::fd_i2c = -1 [private]
```

7.2.4.5 ISR_TIMEOUT_MS

```
constexpr int64_t ADS1115rpi::ISR_TIMEOUT_MS = 500 [static], [constexpr], [private]
```

7.2.4.6 reg_config

```
const uint8_t ADS1115rpi::reg_config = 1 [private]
```

7.2.4.7 reg_hi_thres

```
const uint8_t ADS1115rpi::reg_hi_thres = 3 [private]
```

7.2.4.8 reg_lo_thres

```
const uint8_t ADS1115rpi::reg_lo_thres = 2 [private]
```

7.2.4.9 request

```
std::shared_ptr<gpiod::line_request> ADS1115rpi::request  [private]
```

7.2.4.10 retry_enabled_

```
std::atomic<bool> ADS1115rpi::retry_enabled_ {false}  [private]
```

7.2.4.11 running

```
bool ADS1115rpi::running = false  [private]
```

7.2.4.12 thr

```
std::thread ADS1115rpi::thr  [private]
```

The documentation for this class was generated from the following files:

- src/libs/Piezo/Include/ads1115rpi.h
- src/libs/Piezo/ads1115rpi.cpp

7.3 ADS1115settings Struct Reference

```
#include <ads1115rpi.h>
```

Public Types

- enum SamplingRates {
  FS8HZ = 0 , FS16HZ = 1 , FS32HZ = 2 , FS64HZ = 3 ,
  FS128HZ = 4 , FS250HZ = 5 , FS475HZ = 6 , FS860HZ = 7 }
- enum PGA { FSR2_048 = 2 , FSR1_024 = 3 , FSR0_512 = 4 , FSR0_256 = 5 }
- enum Input { AIN0 = 0 , AIN1 = 1 , AIN2 = 2 , AIN3 = 3 }

Public Member Functions

- unsigned getSamplingRate () const

Public Attributes

- int i2c_bus = 1
- uint8_t address = DEFAULT_ADS1115_ADDRESS
- SamplingRates samplingRate = FS128HZ
- PGA pgaGain = FSR2_048
- Input channel = AIN0
- int drdy_chip = 0
- int drdy_gpio = DEFAULT_ALERT_RDY_TO_GPIO

Static Public Attributes

- static constexpr uint8_t DEFAULT_ADS1115_ADDRESS = 0x48
- static constexpr int DEFAULT_ALERT_RDY_TO_GPIO = 26

7.3.1 Member Enumeration Documentation

7.3.1.1 Input

`enum ADS1115settings::Input`

Enumerator

AIN0	
AIN1	
AIN2	
AIN3	

7.3.1.2 PGA

`enum ADS1115settings::PGA`

Enumerator

FSR2_048	
FSR1_024	
FSR0_512	
FSR0_256	

7.3.1.3 SamplingRates

`enum ADS1115settings::SamplingRates`

Enumerator

FS8HZ	
FS16HZ	
FS32HZ	
FS64HZ	
FS128HZ	
FS250HZ	
FS475HZ	
FS860HZ	

7.3.2 Member Function Documentation

7.3.2.1 getSamplingRate()

```
unsigned ADS1115settings::getSamplingRate ( ) const  [inline]
```

7.3.3 Member Data Documentation

7.3.3.1 address

```
uint8_t ADS1115settings::address = DEFAULT_ADS1115_ADDRESS
```

7.3.3.2 channel

```
Input ADS1115settings::channel = AIN0
```

7.3.3.3 DEFAULT_ADS1115_ADDRESS

```
constexpr uint8_t ADS1115settings::DEFAULT_ADS1115_ADDRESS = 0x48  [static], [constexpr]
```

7.3.3.4 DEFAULT_ALERT_RDY_TO_GPIO

```
constexpr int ADS1115settings::DEFAULT_ALERT_RDY_TO_GPIO = 26  [static], [constexpr]
```

7.3.3.5 drdy_chip

```
int ADS1115settings::drdy_chip = 0
```

7.3.3.6 drdy_gpio

```
int ADS1115settings::drdy_gpio = DEFAULT_ALERT_RDY_TO_GPIO
```

7.3.3.7 i2c_bus

```
int ADS1115settings::i2c_bus = 1
```

7.3.3.8 pgaGain

```
PGA ADS1115settings::pgaGain = FSR2_048
```

7.3.3.9 samplingRate

```
SamplingRates ADS1115settings::samplingRate = FS128HZ
```

The documentation for this struct was generated from the following file:

- src/libs/Piezo/Include/ads1115rpi.h

7.4 PositionDetector::Config Struct Reference

```
#include <positiondetector.hpp>
```

Public Attributes

- float refX = 9.88f
- float refY = -0.87f
- float refZ = 0.10f
- float minG = 7.0f
- float maxG = 12.5f
- float minDot = 0.90f
- float refHeadingDeg = 25.0f
- float headingToleranceDeg = 180.0f
- int stabilitySamples = 10

7.4.1 Member Data Documentation

7.4.1.1 headingToleranceDeg

```
float PositionDetector::Config::headingToleranceDeg = 180.0f
```

7.4.1.2 maxG

```
float PositionDetector::Config::maxG = 12.5f
```

7.4.1.3 minDot

```
float PositionDetector::Config::minDot = 0.90f
```

7.4.1.4 minG

```
float PositionDetector::Config::minG = 7.0f
```

7.4.1.5 refHeadingDeg

```
float PositionDetector::Config::refHeadingDeg = 25.0f
```

7.4.1.6 refX

```
float PositionDetector::Config::refX = 9.88f
```

7.4.1.7 refY

```
float PositionDetector::Config::refY = -0.87f
```

7.4.1.8 refZ

```
float PositionDetector::Config::refZ = 0.10f
```

7.4.1.9 stabilitySamples

```
int PositionDetector::Config::stabilitySamples = 10
```

The documentation for this struct was generated from the following file:

- src/libs/IMU_math/Include/positiondetector.hpp

7.5 icm20948::gyro_settings Struct Reference

```
#include <Icm20948driver.hpp>
```

Public Member Functions

- gyro_settings (uint8_t sample_rate_div=0, gyro_scale scale=GYRO_250DPS, bool dlpf_enable=true, gyro_dlpf_config dlpf_config=GYRO_DLPF_196_6HZ)

Public Attributes

- uint8_t sample_rate_div
- gyro_scale scale
- bool dlpf_enable
- gyro_dlpf_config dlpf_config

7.5.1 Constructor & Destructor Documentation

7.5.1.1 gyro_settings()

```
icm20948::gyro_settings::gyro_settings (
            uint8_t sample_rate_div = 0,
            gyro_scale scale = GYRO_250DPS,
            bool dlpf_enable = true,
            gyro_dlpf_config dlpf_config = GYRO_DLPF_196_6HZ )  [inline]
```

7.5.2 Member Data Documentation

7.5.2.1 dlpf_config

```
gyro_dlpf_config icm20948::gyro_settings::dlpf_config
```

7.5.2.2 dlpf_enable

```
bool icm20948::gyro_settings::dlpf_enable
```

7.5.2.3 sample_rate_div

```
uint8_t icm20948::gyro_settings::sample_rate_div
```

7.5.2.4 scale

```
gyro_scale icm20948::gyro_settings::scale
```

The documentation for this struct was generated from the following file:

- src/libs/IMU/Include/Icm20948driver.hpp

7.6 icm20948::ICM20948_I2C Class Reference

```
#include <Icm20948driver.hpp>
```

Collaboration diagram for icm20948::ICM20948_I2C:

Public Member Functions

- ICM20948_I2C (unsigned i2c_bus, unsigned i2c_address=ICM20948_I2C_ADDR, icm20948::settings settings=icm20948::settings())
- bool init ()
- bool reset ()
- bool wake ()
- bool set_settings ()
- bool read_accel_gyro ()
- bool read_magn ()
- bool read_sample (IMUSample &sample)
- bool check_DRDY_INT ()

Public Attributes

- float accel [3]
- float gyro [3]
- float magn [3]
- icm20948::settings settings

Private Member Functions

- bool _enable_data_ready_interrupt ()
- bool _set_bank (uint8_t bank)
- bool _set_accel_sample_rate_div ()
- bool _set_accel_range_dlpf ()
- bool _set_gyro_sample_rate_div ()
- bool _set_gyro_range_dlpf ()
- bool _magnetometer_init ()
- bool _magnetometer_enable ()
- bool _magnetometer_set_mode ()
- bool _magnetometer_configured ()
- bool _magnetometer_set_readout ()
- bool _chip_i2c_master_reset ()
- bool _write_byte (uint8_t bank, uint8_t reg, uint8_t byte)
- bool _read_byte (uint8_t bank, uint8_t reg, uint8_t &byte)
- bool _write_bit (uint8_t bank, uint8_t reg, uint8_t bit_pos, bool bit)
- bool _read_bit (uint8_t bank, uint8_t reg, uint8_t bit_pos, bool &bit)
- bool _read_block_bytes (uint8_t bank, uint8_t start_reg, uint8_t *bytes, int length)
- bool _write_mag_byte (uint8_t mag_reg, uint8_t byte)
- bool _read_mag_byte (uint8_t mag_reg, uint8_t &byte)

Private Attributes

- int _i2c_fd
- unsigned _i2c_bus
- unsigned _i2c_address
- uint8_t _current_bank
- float _accel_scale_factor
- float _gyro_scale_factor
- float _magn_scale_factor

7.6.1 Constructor & Destructor Documentation

7.6.1.1 ICM20948_I2C()

```
icm20948::ICM20948_I2C::ICM20948_I2C (
            unsigned i2c_bus,
            unsigned i2c_address = ICM20948_I2C_ADDR,
            icm20948::settings settings = icm20948::settings() )
```

7.6.2 Member Function Documentation

7.6.2.1 _chip_i2c_master_reset()

```
bool icm20948::ICM20948_I2C::_chip_i2c_master_reset ( ) [private]
```

7.6.2.2 _enable_data_ready_interrupt()

```
bool icm20948::ICM20948_I2C::_enable_data_ready_interrupt ( ) [private]
```

7.6.2.3 _magnetometer_configured()

```
bool icm20948::ICM20948_I2C::_magnetometer_configured ( ) [private]
```

7.6.2.4 _magnetometer_enable()

```
bool icm20948::ICM20948_I2C::_magnetometer_enable ( ) [private]
```

7.6.2.5 _magnetometer_init()

```
bool icm20948::ICM20948_I2C::_magnetometer_init ( ) [private]
```

7.6.2.6 _magnetometer_set_mode()

```
bool icm20948::ICM20948_I2C::_magnetometer_set_mode ( ) [private]
```

7.6.2.7 _magnetometer_set_readout()

```
bool icm20948::ICM20948_I2C::_magnetometer_set_readout ( ) [private]
```

7.6.2.8 _read_bit()

```
bool icm20948::ICM20948_I2C::_read_bit (
            uint8_t bank,
            uint8_t reg,
            uint8_t bit_pos,
            bool & bit ) [private]
```

7.6.2.9 _read_block_bytes()

```
bool icm20948::ICM20948_I2C::_read_block_bytes (
            uint8_t bank,
            uint8_t start_reg,
            uint8_t * bytes,
            int length ) [private]
```

7.6.2.10 _read_byte()

```
bool icm20948::ICM20948_I2C::_read_byte (
            uint8_t bank,
            uint8_t reg,
            uint8_t & byte )  [private]
```

7.6.2.11 _read_mag_byte()

```
bool icm20948::ICM20948_I2C::_read_mag_byte (
            uint8_t mag_reg,
            uint8_t & byte )  [private]
```

7.6.2.12 _set_accel_range_dlpf()

```
bool icm20948::ICM20948_I2C::_set_accel_range_dlpf ( )  [private]
```

7.6.2.13 _set_accel_sample_rate_div()

```
bool icm20948::ICM20948_I2C::_set_accel_sample_rate_div ( )  [private]
```

7.6.2.14 _set_bank()

```
bool icm20948::ICM20948_I2C::_set_bank (
            uint8_t bank )  [private]
```

7.6.2.15 _set_gyro_range_dlpf()

```
bool icm20948::ICM20948_I2C::_set_gyro_range_dlpf ( )  [private]
```

7.6.2.16 _set_gyro_sample_rate_div()

```
bool icm20948::ICM20948_I2C::_set_gyro_sample_rate_div ( )  [private]
```

7.6.2.17 _write_bit()

```
bool icm20948::ICM20948_I2C::_write_bit (
            uint8_t bank,
            uint8_t reg,
            uint8_t bit_pos,
            bool bit )  [private]
```

7.6.2.18 _write_byte()

```
bool icm20948::ICM20948_I2C::_write_byte (
            uint8_t bank,
            uint8_t reg,
            uint8_t byte )  [private]
```

7.6.2.19 _write_mag_byte()

```
bool icm20948::ICM20948_I2C::_write_mag_byte (
            uint8_t mag_reg,
            uint8_t byte )  [private]
```

7.6.2.20 check_DRDY_INT()

```
bool icm20948::ICM20948_I2C::check_DRDY_INT ( )
```

7.6.2.21 init()

```
bool icm20948::ICM20948_I2C::init ( )
```

7.6.2.22 read_accel_gyro()

```
bool icm20948::ICM20948_I2C::read_accel_gyro ( )
```

7.6.2.23 read_magn()

```
bool icm20948::ICM20948_I2C::read_magn ( )
```

7.6.2.24 read_sample()

```
bool icm20948::ICM20948_I2C::read_sample (
            IMUSample & sample )
```

7.6.2.25 reset()

```
bool icm20948::ICM20948_I2C::reset ( )
```

7.6.2.26 set_settings()

```
bool icm20948::ICM20948_I2C::set_settings ( )
```

7.6.2.27 wake()

```
bool icm20948::ICM20948_I2C::wake ( )
```

7.6.3 Member Data Documentation

7.6.3.1 _accel_scale_factor

```
float icm20948::ICM20948_I2C::_accel_scale_factor [private]
```

7.6.3.2 _current_bank

```
uint8_t icm20948::ICM20948_I2C::_current_bank [private]
```

7.6.3.3 _gyro_scale_factor

```
float icm20948::ICM20948_I2C::_gyro_scale_factor [private]
```

7.6.3.4 _i2c_address

```
unsigned icm20948::ICM20948_I2C::_i2c_address [private]
```

7.6.3.5 _i2c_bus

```
unsigned icm20948::ICM20948_I2C::_i2c_bus [private]
```

7.6.3.6 _i2c_fd

```
int icm20948::ICM20948_I2C::_i2c_fd [private]
```

7.6.3.7 _magn_scale_factor

```
float icm20948::ICM20948_I2C::_magn_scale_factor [private]
```

7.6.3.8 accel

```
float icm20948::ICM20948_I2C::accel[3]
```

7.6.3.9 gyro

```
float icm20948::ICM20948_I2C::gyro[3]
```

7.6.3.10 magn

```
float icm20948::ICM20948_I2C::magn[3]
```

7.6.3.11 settings

```
icm20948::settings icm20948::ICM20948_I2C::settings
```

The documentation for this class was generated from the following files:

- src/libs/IMU/Include/Icm20948driver.hpp
- src/libs/IMU/Icm20948driver.cpp

7.7 IMUReader Class Reference

```
#include <imureader.hpp>
```

Collaboration diagram for IMUReader:

Public Types

- using SampleCallback = std::function< void(float ax, float ay, float az, float gx, float gy, float gz, float mx, float my)>

Public Member Functions

- IMUReader (unsigned i2c_bus, unsigned i2c_address=ICM20948_I2C_ADDR, const char *gpiochip="/dev/gpiochip0", unsigned rdy_line=27, icm20948::settings settings=icm20948::settings())
- ~IMUReader ()
- bool init ()
- void setCallback (SampleCallback cb)
- bool start ()
- void stop ()
- icm20948::ICM20948_I2C & getIMU ()

Private Member Functions

- void worker ()

Private Attributes

- icm20948::ICM20948_I2C imu_
- SampleCallback callback_
- std::thread worker_thread_
- std::atomic_bool running_
- const char * gpiochip_path_
- unsigned rdy_line_
- gpiod_chip * chip_
- gpiod_line_request * request_
- int gpio_fd_
- int stop_fd_

7.7.1 Member Typedef Documentation

7.7.1.1 SampleCallback

```
using IMUReader::SampleCallback = std::function<void(float ax, float ay, float az, float gx,
float gy, float gz, float mx, float my)>
```

7.7.2 Constructor & Destructor Documentation

7.7.2.1 IMUReader()

```
IMUReader::IMUReader (
            unsigned i2c_bus,
            unsigned i2c_address = ICM20948_I2C_ADDR,
            const char * gpiochip = "/dev/gpiochip0",
            unsigned rdy_line = 27,
            icm20948::settings settings = icm20948::settings() )
```

7.7.2.2 ~IMUReader()

```
IMUReader::~IMUReader ( )
```

7.7.3 Member Function Documentation

7.7.3.1 getIMU()

```
icm20948::ICM20948_I2C & IMUReader::getIMU ( )
```

7.7.3.2 init()

```
bool IMUReader::init ( )
```

7.7.3.3 setCallback()

```
void IMUReader::setCallback (
            SampleCallback cb )
```

7.7.3.4 start()

```
bool IMUReader::start ( )
```

7.7.3.5 stop()

```
void IMUReader::stop ( )
```

7.7.3.6 worker()

```
void IMUReader::worker ( ) [private]
```

7.7.4 Member Data Documentation

7.7.4.1 callback_

```
SampleCallback IMUReader::callback_ [private]
```

7.7.4.2 chip_

```
gpiod_chip* IMUReader::chip_ [private]
```

7.7.4.3 gpio_fd_

```
int IMUReader::gpio_fd_ [private]
```

7.7.4.4 gpiochip_path_

```
const char* IMUReader::gpiochip_path_ [private]
```

7.7.4.5 imu_

```
icm20948::ICM20948_I2C IMUReader::imu_ [private]
```

7.7.4.6 rdy_line_

```
unsigned IMUReader::rdy_line_ [private]
```

7.7.4.7 request_

```
gpiod_line_request* IMUReader::request_ [private]
```

7.7.4.8 running_

```
std::atomic_bool IMUReader::running_ [private]
```

7.7.4.9 stop_fd_

```
int IMUReader::stop_fd_ [private]
```

7.7.4.10 worker_thread_

```
std::thread IMUReader::worker_thread_ [private]
```

The documentation for this class was generated from the following files:

- src/libs/IMU/Include/imureader.hpp
- src/libs/IMU/imureader.cpp

7.8 icm20948::IMUSample Struct Reference

```
#include <Icm20948driver.hpp>
```

Public Attributes

- float ax
- float ay
- float az
- float gx
- float gy
- float gz
- float mx
- float my
- float mz

7.8.1 Member Data Documentation

7.8.1.1 ax

```
float icm20948::IMUSample::ax
```

7.8.1.2 ay

```
float icm20948::IMUSample::ay
```

7.8.1.3 az

```
float icm20948::IMUSample::az
```

7.8.1.4 gx

```
float icm20948::IMUSample::gx
```

7.8.1.5 gy

```
float icm20948::IMUSample::gy
```

7.8.1.6 gz

```
float icm20948::IMUSample::gz
```

7.8.1.7 mx

```
float icm20948::IMUSample::mx
```

7.8.1.8 my

```
float icm20948::IMUSample::my
```

7.8.1.9 mz

```
float icm20948::IMUSample::mz
```

The documentation for this struct was generated from the following file:

- src/libs/IMU/Include/Icm20948driver.hpp

7.9 LEDController Class Reference

Declares the LED controller class and its main functions for setting, switching, flashing, and updating the LED output.

```
#include <led_controller.h>
```

Inheritance diagram for LEDController:

Collaboration diagram for LEDController:

Public Member Functions

- LEDController (LEDControllerSettings settings=LEDControllerSettings())

  Initialises the LED controller, clears the LED state, and updates the output.
- ~LEDController () override

  Turns all LED output off when the controller is destroyed.
- void set (bool on) override

  Sets the green LED state directly and updates the output.
- void greenOn ()

  Turns the green LED on and clears any flash timing.
- void greenOff ()

  Turns the green LED off and clears any flash timing.
- void allOff ()

  Turns all LED output off and clears any flash timing.
- void flashGreen (int flashMs)

  Turns the green LED on for a set time and stores when it should switch off.
- void service ()

  Checks whether the flash time has ended and switches the LED off if needed.

Public Member Functions inherited from SimpleLEDController

- SimpleLEDController (int pinNo, int chipNo=0)

  Construct the controller with the GPIO pin number and optional chip number.
- virtual ~SimpleLEDController ()

  Virtual destructor ensures correct cleanup through the base-class interface,.

Private Member Functions

- void updateOutputs ()

  Declares the internal update function and private members used to track the green LED state and flash timing.

Private Attributes

- bool greenActive_ = false
- std::chrono::steady_clock::time_point greenOffTime_ {}

Additional Inherited Members

Protected Member Functions inherited from SimpleLEDController

- SimpleLEDController ()=default

  Protected default constructor allows derived classes controlled access.

Protected Attributes inherited from SimpleLEDController

- int pin = -1

  Internal GPIO configuration state is protected so it is available to.
- int chipNumber = -1
- std::shared_ptr< gpiod::chip > chip

  Shared ownership is used for the GPIO chip/request resources so that the.
- std::shared_ptr< gpiod::line_request > request

7.9.1 Detailed Description

Declares the LED controller class and its main functions for setting, switching, flashing, and updating the LED output.

7.9.2 Constructor & Destructor Documentation

7.9.2.1 LEDController()

```
LEDController::LEDController (
            LEDControllerSettings settings = LEDControllerSettings() )
```

Initialises the LED controller, clears the LED state, and updates the output.

7.9.2.2 ~LEDController()

```
LEDController::~LEDController ( ) [override]
```

Turns all LED output off when the controller is destroyed.

7.9.3 Member Function Documentation

7.9.3.1 allOff()

```
void LEDController::allOff ( )
```

Turns all LED output off and clears any flash timing.

7.9.3.2 flashGreen()

```
void LEDController::flashGreen (
            int flashMs ) [virtual]
```

Turns the green LED on for a set time and stores when it should switch off.

Reimplemented from SimpleLEDController.

7.9.3.3 greenOff()

```
void LEDController::greenOff ( )
```

Turns the green LED off and clears any flash timing.

7.9.3.4 greenOn()

```
void LEDController::greenOn ( )
```

Turns the green LED on and clears any flash timing.

7.9.3.5 service()

```
void LEDController::service ( ) [virtual]
```

Checks whether the flash time has ended and switches the LED off if needed.

Reimplemented from SimpleLEDController.

7.9.3.6 set()

```
void LEDController::set (
            bool on )  [override], [virtual]
```

Sets the green LED state directly and updates the output.

Reimplemented from SimpleLEDController.

7.9.3.7 updateOutputs()

```
void LEDController::updateOutputs ( )  [private]
```

Declares the internal update function and private members used to track the green LED state and flash timing.

7.9.4 Member Data Documentation

7.9.4.1 greenActive_

```
bool LEDController::greenActive_ = false  [private]
```

7.9.4.2 greenOffTime_

```
std::chrono::steady_clock::time_point LEDController::greenOffTime_ {}  [private]
```

The documentation for this class was generated from the following files:

- src/libs/LEDs/Include/led_controller.h
- src/libs/LEDs/led_controller.cpp

7.10 LEDControllerSettings Struct Reference

This header declares the LED controller settings and class.

```
#include <led_controller.h>
```

Public Attributes

- int chipNumber = 0
- unsigned int greenGpio = 25

7.10.1 Detailed Description

This header declares the LED controller settings and class.

It manages the green LED output, supports timed flashing,

and updates the LED state through the inherited controller interface.

Declares the LED controller settings, including the chip number and green LED GPIO pin.

7.10.2 Member Data Documentation

7.10.2.1 chipNumber

```
int LEDControllerSettings::chipNumber = 0
```

7.10.2.2 greenGpio

```
unsigned int LEDControllerSettings::greenGpio = 25
```

The documentation for this struct was generated from the following file:

- src/libs/LEDs/Include/led_controller.h

7.11 icm20948::magn_settings Struct Reference

```
#include <Icm20948driver.hpp>
```

Public Member Functions

- magn_settings (magn_mode mode=MAGN_100HZ)

Public Attributes

- magn_mode mode

7.11.1 Constructor & Destructor Documentation

7.11.1.1 magn_settings()

```
icm20948::magn_settings::magn_settings (
            magn_mode mode = MAGN_100HZ )  [inline], [explicit]
```

7.11.2 Member Data Documentation

7.11.2.1 mode

```
magn_mode icm20948::magn_settings::mode
```

The documentation for this struct was generated from the following file:

- src/libs/IMU/Include/Icm20948driver.hpp

7.12 PiezoEventDetector Class Reference

```
#include <event_detector.h>
```

Collaboration diagram for PiezoEventDetector:

Public Types

- using ForceCallback = std::function< void(bool)>

Public Member Functions

- PiezoEventDetector (SimpleLEDController &ledRef, PiezoEventDetectorSettings settings)
- void setForceCallback (ForceCallback cb)
- void processSample (float v)
- void reset ()

Private Attributes

- SimpleLEDController & led_
- PiezoEventDetectorSettings settings_
- ForceCallback forceCallback_
- bool pressed_ = false
- bool filter_init_ = false
- float filtered_v_ = 0.0f

7.12.1 Member Typedef Documentation

7.12.1.1 ForceCallback

```
using PiezoEventDetector::ForceCallback = std::function<void(bool)>
```

7.12.2 Constructor & Destructor Documentation

7.12.2.1 PiezoEventDetector()

```
PiezoEventDetector::PiezoEventDetector (
            SimpleLEDController & ledRef,
            PiezoEventDetectorSettings settings )
```

7.12.3 Member Function Documentation

7.12.3.1 processSample()

```
void PiezoEventDetector::processSample (
            float v )
```

7.12.3.2 reset()

```
void PiezoEventDetector::reset ( )
```

7.12.3.3 setForceCallback()

```
void PiezoEventDetector::setForceCallback (
            ForceCallback cb )
```

7.12.4 Member Data Documentation

7.12.4.1 filter_init_

```
bool PiezoEventDetector::filter_init_ = false  [private]
```

7.12.4.2 filtered_v_

```
float PiezoEventDetector::filtered_v_ = 0.0f  [private]
```

7.12.4.3 forceCallback_

```
ForceCallback PiezoEventDetector::forceCallback_  [private]
```

7.12.4.4 led_

```
SimpleLEDController& PiezoEventDetector::led_  [private]
```

7.12.4.5 pressed_

```
bool PiezoEventDetector::pressed_ = false  [private]
```

7.12.4.6 settings_

```
PiezoEventDetectorSettings PiezoEventDetector::settings_  [private]
```

The documentation for this class was generated from the following files:

- src/libs/Piezo/Include/event_detector.h
- src/libs/Piezo/event_detector.cpp

7.13 PiezoEventDetectorSettings Struct Reference

```
#include <event_detector.h>
```

Public Attributes

- float pressThreshold = 0.95f
- float releaseThreshold = 0.70f
- bool enableDebugPrints = false

7.13.1 Member Data Documentation

7.13.1.1 enableDebugPrints

```
bool PiezoEventDetectorSettings::enableDebugPrints = false
```

7.13.1.2 pressThreshold

```
float PiezoEventDetectorSettings::pressThreshold = 0.95f
```

7.13.1.3 releaseThreshold

```
float PiezoEventDetectorSettings::releaseThreshold = 0.70f
```

The documentation for this struct was generated from the following file:

- src/libs/Piezo/Include/event_detector.h

7.14 PositionDetector Class Reference

```
#include <positiondetector.hpp>
```

Inheritance diagram for PositionDetector:

Collaboration diagram for PositionDetector:

Classes

- struct Config

Public Types

- using StateCallback = std::function< void(bool upright)>

Public Member Functions

- PositionDetector ()
- PositionDetector (const Config &c)
- virtual ~PositionDetector ()=default
- virtual void onSample (float ax, float ay, float az, float mx, float my)
- void setCallback (StateCallback cb)
- void resetState ()

Static Private Member Functions

- static float wrapAngleDeg (float a)

Private Attributes

- Config cfg
- float nRefX
- float nRefY
- float nRefZ
- int stableCount
- bool confirmed
- StateCallback callback

7.14.1 Member Typedef Documentation

7.14.1.1 StateCallback

```
using PositionDetector::StateCallback = std::function<void(bool upright)>
```

7.14.2 Constructor & Destructor Documentation

7.14.2.1 PositionDetector() [1/2]

```
PositionDetector::PositionDetector ( )
```

7.14.2.2 PositionDetector() [2/2]

```
PositionDetector::PositionDetector (
            const Config & c )  [explicit]
```

7.14.2.3 ~PositionDetector()

```
virtual PositionDetector::~PositionDetector ( )  [virtual], [default]
```

7.14.3 Member Function Documentation

7.14.3.1 onSample()

```
void PositionDetector::onSample (
            float ax,
            float ay,
            float az,
            float mx,
            float my )  [virtual]
```

Reimplemented in SwingProcessor.

7.14.3.2 resetState()

```
void PositionDetector::resetState ( )
```

7.14.3.3 setCallback()

```
void PositionDetector::setCallback (
            StateCallback cb )  [inline]
```

7.14.3.4 wrapAngleDeg()

```
float PositionDetector::wrapAngleDeg (
            float a )  [static], [private]
```

7.14.4 Member Data Documentation

7.14.4.1 callback

```
StateCallback PositionDetector::callback [private]
```

7.14.4.2 cfg

```
Config PositionDetector::cfg [private]
```

7.14.4.3 confirmed

```
bool PositionDetector::confirmed [private]
```

7.14.4.4 nRefX

```
float PositionDetector::nRefX [private]
```

7.14.4.5 nRefY

```
float PositionDetector::nRefY [private]
```

7.14.4.6 nRefZ

```
float PositionDetector::nRefZ [private]
```

7.14.4.7 stableCount

```
int PositionDetector::stableCount [private]
```

The documentation for this class was generated from the following files:

- src/libs/IMU_math/Include/positiondetector.hpp
- src/libs/IMU_math/positiondetector.cpp

7.15 RPI_PWM Class Reference

```
#include <rpi_pwm.h>
```

Public Member Functions

- RPI_PWM ()
- bool start ()
- void stop ()
- void setDutyCycle (float percent)
- float getDutyCycle () const
- ~RPI_PWM ()

Private Member Functions

- void waitUs (int tfd, int us)
- void loop ()

Static Private Member Functions

- static itimerspec usToTimerspec (long us)

Private Attributes

- std::atomic< float > duty_
- std::atomic< bool > running_
- std::thread thread_
- gpiod_chip * chip_
- gpiod_line_request * request_

7.15.1 Constructor & Destructor Documentation

7.15.1.1 RPI_PWM()

```
RPI_PWM::RPI_PWM ( ) [inline]
```

7.15.1.2 ~RPI_PWM()

```
RPI_PWM::~RPI_PWM ( ) [inline]
```

7.15.2 Member Function Documentation

7.15.2.1 getDutyCycle()

```
float RPI_PWM::getDutyCycle ( ) const [inline]
```

7.15.2.2 loop()

```
void RPI_PWM::loop ( ) [inline], [private]
```

7.15.2.3 setDutyCycle()

```
void RPI_PWM::setDutyCycle (
            float percent )  [inline]
```

7.15.2.4 start()

```
bool RPI_PWM::start ( )  [inline]
```

7.15.2.5 stop()

```
void RPI_PWM::stop ( )  [inline]
```

7.15.2.6 usToTimerspec()

```
static itimerspec RPI_PWM::usToTimerspec (
            long us )  [inline], [static], [private]
```

7.15.2.7 waitUs()

```
void RPI_PWM::waitUs (
            int tfd,
            int us )  [inline], [private]
```

7.15.3 Member Data Documentation

7.15.3.1 chip_

```
gpiod_chip* RPI_PWM::chip_  [private]
```

7.15.3.2 duty_

```
std::atomic<float> RPI_PWM::duty_  [private]
```

7.15.3.3 request_

```
gpiod_line_request* RPI_PWM::request_  [private]
```

7.15.3.4 running_

```
std::atomic<bool> RPI_PWM::running_  [private]
```

7.15.3.5 thread_

```
std::thread RPI_PWM::thread_  [private]
```

The documentation for this class was generated from the following file:

- src/libs/Motor/Include/rpi_pwm.h

7.16 icm20948::settings Struct Reference

```
#include <Icm20948driver.hpp>
```

Collaboration diagram for icm20948::settings:

Public Member Functions

- settings (accel_settings accel=accel_settings(), gyro_settings gyro=gyro_settings(), magn_settings magn=magn_settings())
- settings (YAML::Node config_file_node)

Public Attributes

- accel_settings accel
- gyro_settings gyro
- magn_settings magn

7.16.1 Constructor & Destructor Documentation

7.16.1.1 settings() [1/2]

```
icm20948::settings::settings (
            accel_settings accel = accel_settings(),
            gyro_settings gyro = gyro_settings(),
            magn_settings magn = magn_settings() )  [inline]
```

7.16.1.2 settings() [2/2]

```
icm20948::settings::settings (
            YAML::Node config_file_node )  [explicit]
```

7.16.2 Member Data Documentation

7.16.2.1 accel

```
accel_settings icm20948::settings::accel
```

7.16.2.2 gyro

```
gyro_settings icm20948::settings::gyro
```

7.16.2.3 magn

```
magn_settings icm20948::settings::magn
```

The documentation for this struct was generated from the following files:

- src/libs/IMU/Include/Icm20948driver.hpp
- src/libs/IMU/Icm20948driver.cpp

7.17 SimpleLEDController Class Reference

This header declares a simple LED controller base class. This is to control the LED associated with the IMU swing position.

```
#include <ledcallback.hpp>
```

Inheritance diagram for SimpleLEDController:

Public Member Functions

- SimpleLEDController (int pinNo, int chipNo=0)

  Construct the controller with the GPIO pin number and optional chip number.
- virtual ~SimpleLEDController ()

  Virtual destructor ensures correct cleanup through the base-class interface,.
- virtual void set (bool on)

  Minimal public LED interface: callers express intent (on/off) while the.
- virtual void flashGreen (int)

  Optional extension point for derived classes that support timed flashing.
- virtual void service ()

  Optional extension point for derived classes that need periodic servicing,.

Protected Member Functions

- SimpleLEDController ()=default

  Protected default constructor allows derived classes controlled access.

Protected Attributes

- int pin = -1

  Internal GPIO configuration state is protected so it is available to.
- int chipNumber = -1
- std::shared_ptr< gpiod::chip > chip

  Shared ownership is used for the GPIO chip/request resources so that the.
- std::shared_ptr< gpiod::line_request > request

7.17.1 Detailed Description

This header declares a simple LED controller base class. This is to control the LED associated with the IMU swing position.

Design intention:

- Single Responsibility:

This class is responsible only for wrapping the basic GPIO-backed LED

control mechanism and exposing a small interface for derived LED classes.

- Open/Closed:

Derived classes can extend behaviour such as flashing or timed service

updates without changing the base class interface.

- Liskov Substitution:

Client code can use this base class interface and substitute any derived

LED controller that implements the same contract.

- Interface Segregation / Dependency Inversion:

Other parts of the system can depend on this small LED abstraction rather

than on a larger concrete GPIO implementation.

It provides:

- basic LED control through set(bool),
- optional flash/service hooks for derived classes,
- storage for the GPIO resources needed to control the LED.

SimpleLEDController acts as a narrow abstraction for LED behaviour.

Higher-level logic can depend on this interface rather than depending

directly on a concrete GPIO-specific LED implementation.

7.17.2 Constructor & Destructor Documentation

7.17.2.1 SimpleLEDController() [1/2]

```
SimpleLEDController::SimpleLEDController (
            int pinNo,
            int chipNo = 0 )
```

Construct the controller with the GPIO pin number and optional chip number.

This file implements the simple LED controller class.

The dependency on the physical GPIO configuration is contained here so that

client code does not need to manage those hardware details itself.

It sets up the GPIO line for LED output, releases the GPIO resources

when finished, and allows the LED state to be switched on or off.

Initialises the LED controller by setting up the GPIO chip, output line, and line request for the selected pin.

7.17.2.2 ~SimpleLEDController()

```
SimpleLEDController::~SimpleLEDController ( )  [virtual]
```

Virtual destructor ensures correct cleanup through the base-class interface,.

Releases the GPIO resources and makes sure the LED is turned off before closing.

supporting safe substitution of derived LED controller types.

7.17.2.3 SimpleLEDController() [2/2]

```
SimpleLEDController::SimpleLEDController ( )  [protected], [default]
```

Protected default constructor allows derived classes controlled access.

without exposing incomplete construction to general client code.

7.17.3 Member Function Documentation

7.17.3.1 flashGreen()

```
virtual void SimpleLEDController::flashGreen (
            int  )  [inline], [virtual]
```

Optional extension point for derived classes that support timed flashing.

Default implementation does nothing so derived classes are not forced to

implement behaviour they do not need.

Reimplemented in LEDController.

7.17.3.2 service()

```
virtual void SimpleLEDController::service ( ) [inline], [virtual]
```

Optional extension point for derived classes that need periodic servicing,.

such as time-based LED state updates.

Reimplemented in LEDController.

7.17.3.3 set()

```
void SimpleLEDController::set (
            bool on ) [virtual]
```

Minimal public LED interface: callers express intent (on/off) while the.

Sets the LED output state to on or off.

hardware-specific details remain encapsulated inside the implementation.

Reimplemented in LEDController.

7.17.4 Member Data Documentation

7.17.4.1 chip

```
std::shared_ptr<gpiod::chip> SimpleLEDController::chip [protected]
```

Shared ownership is used for the GPIO chip/request resources so that the.

resource lifetime is managed safely without manual memory handling.

7.17.4.2 chipNumber

```
int SimpleLEDController::chipNumber = -1 [protected]
```

7.17.4.3 pin

```
int SimpleLEDController::pin = -1 [protected]
```

Internal GPIO configuration state is protected so it is available to.

derived implementations, while still hidden from external users.

7.17.4.4 request

```
std::shared_ptr<gpiod::line_request> SimpleLEDController::request [protected]
```

The documentation for this class was generated from the following files:

- src/libs/LEDs/Include/ledcallback.hpp
- src/libs/LEDs/ledcallback.cpp

7.18 SwingCalibrator Class Reference

```
#include <SwingCalibrator.hpp>
```

Public Types

- using DoneCallback = std::function< void(float bx, float by, float bz, bool initial)>

Public Member Functions

- SwingCalibrator (int initial_samples=200, int recal_samples=50)
- void setCallback (DoneCallback cb)
- bool feed (float ax, float ay, float az)
- void triggerRecal ()

  Start a recalibration pass if not already calibrating.
- bool isReady () const
- float biasX () const
- float biasY () const
- float biasZ () const

Private Types

- enum class State { INITIAL , READY , RECAL }

Private Member Functions

- void accumulate (float ax, float ay, float az, int target, bool initial)

  Accumulate one sample and finalise when target samples are reached.

Private Attributes

- int initial_samples_
- int recal_samples_
- State state_
- int count_
- float sum_x_

  Running sums for the current calibration window.
- float sum_y_
- float sum_z_
- float bias_x_

  Latest completed bias estimate.
- float bias_y_
- float bias_z_
- DoneCallback callback_

7.18.1 Detailed Description

Gravity-bias calibrator for the swing detector. Feed IMU samples continuously; while calibrating, feed() returns true. Once ready, biasX/Y/Z() hold the current bias estimate. triggerRecal() starts a short recalibration window.

7.18.2 Member Typedef Documentation

7.18.2.1 DoneCallback

```
using SwingCalibrator::DoneCallback = std::function<void(float bx, float by, float bz, bool
initial)>
```

Called when a calibration pass completes. initial == true for startup calibration, false for recalibration.

7.18.3 Member Enumeration Documentation

7.18.3.1 State

```
enum class SwingCalibrator::State  [strong], [private]
```

Enumerator

INITIAL	
READY	
RECAL	

7.18.4 Constructor & Destructor Documentation

7.18.4.1 SwingCalibrator()

```
SwingCalibrator::SwingCalibrator (
            int initial_samples = 200,
            int recal_samples = 50 )  [explicit]
```

7.18.5 Member Function Documentation

7.18.5.1 accumulate()

```
void SwingCalibrator::accumulate (
            float ax,
            float ay,
            float az,
            int target,
            bool initial )  [private]
```

Accumulate one sample and finalise when target samples are reached.

7.18.5.2 biasX()

```
float SwingCalibrator::biasX ( ) const  [inline]
```

7.18.5.3 biasY()

```
float SwingCalibrator::biasY ( ) const  [inline]
```

7.18.5.4 biasZ()

```
float SwingCalibrator::biasZ ( ) const  [inline]
```

7.18.5.5 feed()

```
bool SwingCalibrator::feed (
            float ax,
            float ay,
            float az )
```

Push one accel sample. Returns true if the sample was consumed for calibration.

7.18.5.6 isReady()

```
bool SwingCalibrator::isReady ( ) const
```

7.18.5.7 setCallback()

```
void SwingCalibrator::setCallback (
            DoneCallback cb )
```

7.18.5.8 triggerRecal()

```
void SwingCalibrator::triggerRecal ( )
```

Start a recalibration pass if not already calibrating.

7.18.6 Member Data Documentation

7.18.6.1 bias_x_

```
float SwingCalibrator::bias_x_  [private]
```

Latest completed bias estimate.

7.18.6.2 bias_y_

```
float SwingCalibrator::bias_y_ [private]
```

7.18.6.3 bias_z_

```
float SwingCalibrator::bias_z_ [private]
```

7.18.6.4 callback_

```
DoneCallback SwingCalibrator::callback_ [private]
```

7.18.6.5 count_

```
int SwingCalibrator::count_ [private]
```

7.18.6.6 initial_samples_

```
int SwingCalibrator::initial_samples_ [private]
```

7.18.6.7 recal_samples_

```
int SwingCalibrator::recal_samples_ [private]
```

7.18.6.8 state_

```
State SwingCalibrator::state_ [private]
```

7.18.6.9 sum_x_

```
float SwingCalibrator::sum_x_ [private]
```

Running sums for the current calibration window.

7.18.6.10 sum_y_

```
float SwingCalibrator::sum_y_ [private]
```

7.18.6.11 sum_z_

```
float SwingCalibrator::sum_z_ [private]
```

The documentation for this class was generated from the following files:

- src/libs/IMU_math/Include/SwingCalibrator.hpp
- src/libs/IMU_math/SwingCalibrator.cpp

7.19 SwingDetector Class Reference

```
#include <SwingDetector.hpp>
```

Public Types

- using EventCallback = std::function< void(const char *level)>

Public Member Functions

- SwingDetector ()
- void setCallback (EventCallback cb)
- void detect (float accel_mag)
- void reset ()

Private Types

- enum class Level { NONE , LOW , MEDIUM , HIGH }

Static Private Member Functions

- static Level classify (float accel_mag)
- static const char * label (Level level)

Private Attributes

- Level last_level_
- EventCallback callback_

7.19.1 Member Typedef Documentation

7.19.1.1 EventCallback

```
using SwingDetector::EventCallback = std::function<void(const char* level)>
```

7.19.2 Member Enumeration Documentation

7.19.2.1 Level

```
enum class SwingDetector::Level [strong], [private]
```

Enumerator

NONE	
LOW	
MEDIUM	
HIGH	

7.19.3 Constructor & Destructor Documentation

7.19.3.1 SwingDetector()

```
SwingDetector::SwingDetector ( )
```

7.19.4 Member Function Documentation

7.19.4.1 classify()

```
SwingDetector::Level SwingDetector::classify (
            float accel_mag )  [static], [private]
```

7.19.4.2 detect()

```
void SwingDetector::detect (
            float accel_mag )
```

7.19.4.3 label()

```
const char * SwingDetector::label (
            Level level )  [static], [private]
```

7.19.4.4 reset()

```
void SwingDetector::reset ( )
```

7.19.4.5 setCallback()

```
void SwingDetector::setCallback (
            EventCallback cb )
```

7.19.5 Member Data Documentation

7.19.5.1 callback_

```
EventCallback SwingDetector::callback_  [private]
```

7.19.5.2 last_level_

```
Level SwingDetector::last_level_ [private]
```

The documentation for this class was generated from the following files:

- src/libs/IMU_math/Include/SwingDetector.hpp
- src/libs/IMU_math/SwingDetector.cpp

7.20 SwingFeedback Class Reference

```
#include <swingfeedback.h>
```

Collaboration diagram for SwingFeedback:

Public Types

- using ResetCallback = std::function< void() >

Public Member Functions

- SwingFeedback (RPI_PWM &pwm)
- void setResetCallback (ResetCallback cb)
- void checkTimeout ()
- void onLevel (const char *level)
- void forceOff ()

Private Member Functions

- void worker ()

Private Attributes

- RPI_PWM & pwm_
- ResetCallback reset_cb_
- std::atomic< bool > busy_ {false}
- std::atomic< int > pending_level_ {0}
- std::atomic< int64_t > busy_since_ms_ {0}

Static Private Attributes

- static constexpr int64_t TIMEOUT_MS = 3000

7.20.1 Member Typedef Documentation

7.20.1.1 ResetCallback

```
using SwingFeedback::ResetCallback = std::function<void()>
```

7.20.2 Constructor & Destructor Documentation

7.20.2.1 SwingFeedback()

```
SwingFeedback::SwingFeedback (
            RPI_PWM & pwm )  [explicit]
```

7.20.3 Member Function Documentation

7.20.3.1 checkTimeout()

```
void SwingFeedback::checkTimeout ( )
```

7.20.3.2 forceOff()

```
void SwingFeedback::forceOff ( )
```

7.20.3.3 onLevel()

```
void SwingFeedback::onLevel (
            const char * level )
```

7.20.3.4 setResetCallback()

```
void SwingFeedback::setResetCallback (
            ResetCallback cb )
```

7.20.3.5 worker()

```
void SwingFeedback::worker ( )  [private]
```

7.20.4 Member Data Documentation

7.20.4.1 busy_

```
std::atomic<bool> SwingFeedback::busy_ {false}  [private]
```

7.20.4.2 busy_since_ms_

```
std::atomic<int64_t> SwingFeedback::busy_since_ms_ {0}  [private]
```

7.20.4.3 pending_level_

```
std::atomic<int> SwingFeedback::pending_level_ {0}  [private]
```

7.20.4.4 pwm_

```
RPI_PWM& SwingFeedback::pwm_  [private]
```

7.20.4.5 reset_cb_

```
ResetCallback SwingFeedback::reset_cb_  [private]
```

7.20.4.6 TIMEOUT_MS

```
constexpr int64_t SwingFeedback::TIMEOUT_MS = 3000  [static], [constexpr], [private]
```

The documentation for this class was generated from the following files:

- src/libs/Motor/Include/swingfeedback.h
- src/libs/Motor/swingfeedback.cpp

7.21 SwingProcessor Class Reference

```
#include <SwingProcessor.hpp>
```

Inheritance diagram for SwingProcessor:

Collaboration diagram for SwingProcessor:

Public Types

- using MagnitudeCallback = std::function< void(float accel_mag)>
- using PositionStateCallback = std::function< void(bool upright)>

Public Types inherited from PositionDetector

- using StateCallback = std::function< void(bool upright)>

Public Member Functions

- SwingProcessor (float accel_deadband=ACCEL_DEADBAND, int calib_samples=CALIB_SAMPLES, int recal_samples=RECAL_SAMPLES)
- void setMagnitudeCallback (MagnitudeCallback cb)
- void setPositionCallback (PositionStateCallback cb)
- void onForceReady (bool correctForce)
- bool isActive () const
- void onPositionReady (bool upright)
- void reset ()
- void onSample (float ax, float ay, float az, float mx, float my) override
- void operator() (float ax, float ay, float az, float gx, float gy, float gz, float mx, float my)

Public Member Functions inherited from PositionDetector

- PositionDetector ()
- PositionDetector (const Config &c)
- virtual ~PositionDetector ()=default
- void setCallback (StateCallback cb)
- void resetState ()

Private Attributes

- float accel_deadband_
- SwingCalibrator calibrator_
- MagnitudeCallback mag_callback_
- PositionStateCallback pos_callback_
- bool isArmed_ = false
- bool isActive_ = false
- std::chrono::steady_clock::time_point reset_time_ {}

Static Private Attributes

- static constexpr int COOLDOWN_MS = 500

7.21.1 Member Typedef Documentation

7.21.1.1 MagnitudeCallback

```
using SwingProcessor::MagnitudeCallback = std::function<void(float accel_mag)>
```

7.21.1.2 PositionStateCallback

```
using SwingProcessor::PositionStateCallback = std::function<void(bool upright)>
```

7.21.2 Constructor & Destructor Documentation

7.21.2.1 SwingProcessor()

```
SwingProcessor::SwingProcessor (
            float accel_deadband = ACCEL_DEADBAND,
            int calib_samples = CALIB_SAMPLES,
            int recal_samples = RECAL_SAMPLES )  [explicit]
```

7.21.3 Member Function Documentation

7.21.3.1 isActive()

```
bool SwingProcessor::isActive ( ) const  [inline]
```

7.21.3.2 onForceReady()

```
void SwingProcessor::onForceReady (
            bool correctForce )
```

7.21.3.3 onPositionReady()

```
void SwingProcessor::onPositionReady (
            bool upright )
```

7.21.3.4 onSample()

```
void SwingProcessor::onSample (
            float ax,
            float ay,
            float az,
            float mx,
            float my )  [override], [virtual]
```

Reimplemented from PositionDetector.

7.21.3.5 operator()()

```
void SwingProcessor::operator() (
            float ax,
            float ay,
            float az,
            float gx,
            float gy,
            float gz,
            float mx,
            float my )
```

7.21.3.6 reset()

```
void SwingProcessor::reset ( )
```

7.21.3.7 setMagnitudeCallback()

```
void SwingProcessor::setMagnitudeCallback (
            MagnitudeCallback cb )
```

7.21.3.8 setPositionCallback()

```
void SwingProcessor::setPositionCallback (
            PositionStateCallback cb )
```

7.21.4 Member Data Documentation

7.21.4.1 accel_deadband_

```
float SwingProcessor::accel_deadband_ [private]
```

7.21.4.2 calibrator_

```
SwingCalibrator SwingProcessor::calibrator_ [private]
```

7.21.4.3 COOLDOWN_MS

```
constexpr int SwingProcessor::COOLDOWN_MS = 500 [static], [constexpr], [private]
```

7.21.4.4 isActive_

```
bool SwingProcessor::isActive_ = false [private]
```

7.21.4.5 isArmed_

```
bool SwingProcessor::isArmed_ = false [private]
```

7.21.4.6 mag_callback_

```
MagnitudeCallback SwingProcessor::mag_callback_ [private]
```

7.21.4.7 pos_callback_

```
PositionStateCallback SwingProcessor::pos_callback_ [private]
```

7.21.4.8 reset_time_

```
std::chrono::steady_clock::time_point SwingProcessor::reset_time_ {} [private]
```

The documentation for this class was generated from the following files:

- src/libs/IMU_math/Include/SwingProcessor.hpp
- src/libs/IMU_math/SwingProcessor.cpp

Chapter 8

File Documentation

8.1 src/libs/IMU/Icm20948driver.cpp File Reference

```
#include "Icm20948driver.hpp"
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <string>
#include <thread>
#include <linux/i2c-dev.h>
#include <linux/i2c.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <cerrno>
#include <cstdio>
#include <cstring>
```

Include dependency graph for Icm20948driver.cpp:

8.2 src/libs/IMU/imureader.cpp File Reference

```
#include "imureader.hpp"
#include "i2c_mutex.hpp"
#include <unistd.h>
#include <sys/epoll.h>
#include <sys/eventfd.h>
#include <gpiod.h>
```

Include dependency graph for imureader.cpp:

8.3 src/libs/IMU/Include/i2c_mutex.hpp File Reference

```
#include <mutex>
```

Include dependency graph for i2c_mutex.hpp: This graph shows which files directly or indirectly include this file:

Functions

- std::mutex & i2c1_mutex ()

8.3.1 Function Documentation

8.3.1.1 i2c1_mutex()

```
std::mutex & i2c1_mutex ( )  [inline]
```

8.4 i2c_mutex.hpp

Go to the documentation of this file.

```
00001 // Shared mutex protecting access to the /dev/i2c-1 bus.
00002 //
00003 // SOLID principles:
00004 //   S - This helper provides one shared synchronisation point for I2C bus access.
00005 //   D - Centralising the bus lock in one place avoids duplicating global mutex
00006 //       definitions across multiple modules and makes cross-thread I2C ownership
00007 //       easier to reason about.
00008
00009 #pragma once
00010 #include <mutex>
00011
00012 // Shared mutex for the I2C-1 bus.
00013 // IMUReader (ICM20948) and ADS1115rpi run on separate threads and both
00014 // access /dev/i2c-1. Without this lock, their ioctl(I2C_SLAVE) and
00015 // read/write sequences can interleave, causing transactions to target
00016 // the wrong device and triggering the ADC retry failures seen in logs.
00017
00018 inline std::mutex& i2c1_mutex()
00019 {
00020     // Function-local static storage gives one shared mutex instance for the
00021     // whole process while keeping the global namespace clean.
00022     static std::mutex m;
00023     return m;
00024 }
```

8.5 src/libs/IMU/Include/Icm20948driver.hpp File Reference

```
#include <cstdint>
#include <string>
#include <linux/i2c-dev.h>
#include <linux/i2c.h>
#include "yaml-cpp/yaml.h"
```

Include dependency graph for Icm20948driver.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct icm20948::accel_settings
- struct icm20948::gyro_settings
- struct icm20948::magn_settings
- struct icm20948::settings
- struct icm20948::IMUSample
- class icm20948::ICM20948_I2C

Namespaces

- namespace icm20948

Macros

- #define ICM20948_I2C_ADDR 0x69
- #define ICM20948_MAGN_I2C_ADDR 0x0C
- #define ICM20948_REG_BANK_SEL_ADDR 0x7F
- #define ICM20948_REG_BANK_SEL_BANK0_VALUE 0x00
- #define ICM20948_REG_BANK_SEL_BANK1_VALUE 0x10
- #define ICM20948_REG_BANK_SEL_BANK2_VALUE 0x20
- #define ICM20948_REG_BANK_SEL_BANK3_VALUE 0x30
- #define ICM20948_WHO_AM_I_ADDR 0x00
- #define ICM20948_USER_CTRL_ADDR 0x03
- #define ICM20948_PWR_MGMT_1_ADDR 0x06
- #define ICM20948_INT_PIN_CFG_ADDR 0x0F
- #define ICM20948_INT_ENABLE_1_ADDR 0x11
- #define ICM20948_I2C_MST_STATUS_ADDR 0x17
- #define ICM20948_ACCEL_XOUT_H_ADDR 0x2D
- #define ICM20948_ACCEL_XOUT_L_ADDR 0x2E
- #define ICM20948_ACCEL_YOUT_H_ADDR 0x2F
- #define ICM20948_ACCEL_YOUT_L_ADDR 0x30
- #define ICM20948_ACCEL_ZOUT_H_ADDR 0x31
- #define ICM20948_ACCEL_ZOUT_L_ADDR 0x32
- #define ICM20948_GYRO_XOUT_H_ADDR 0x33
- #define ICM20948_GYRO_XOUT_L_ADDR 0x34
- #define ICM20948_GYRO_YOUT_H_ADDR 0x35
- #define ICM20948_GYRO_YOUT_L_ADDR 0x36
- #define ICM20948_GYRO_ZOUT_H_ADDR 0x37
- #define ICM20948_GYRO_ZOUT_L_ADDR 0x38
- #define ICM20948_TEMP_OUT_H_ADDR 0x39
- #define ICM20948_TEMP_OUT_L_ADDR 0x3A
- #define ICM20948_EXT_SLV_SENS_DATA_00_ADDR 0x3B
- #define ICM20948_WHO_AM_I_BANK 0
- #define ICM20948_USER_CTRL_BANK 0
- #define ICM20948_PWR_MGMT_1_BANK 0
- #define ICM20948_INT_PIN_CFG_BANK 0
- #define ICM20948_INT_ENABLE_1_BANK 0
- #define ICM20948_I2C_MST_STATUS_BANK 0
- #define ICM20948_ACCEL_OUT_BANK 0
- #define ICM20948_GYRO_OUT_BANK 0
- #define ICM20948_EXT_SLV_SENS_DATA_00_BANK 0
- #define ICM20948_GYRO_SMPLRT_DIV_ADDR 0x00
- #define ICM20948_GYRO_CONFIG_1_ADDR 0x01
- #define ICM20948_ACCEL_SMPLRT_DIV_1_ADDR 0x10
- #define ICM20948_ACCEL_SMPLRT_DIV_2_ADDR 0x11
- #define ICM20948_ACCEL_CONFIG_1_ADDR 0x14
- #define ICM20948_GYRO_SMPLRT_DIV_BANK 2
- #define ICM20948_GYRO_CONFIG_1_BANK 2
- #define ICM20948_ACCEL_SMPLRT_DIV_1_BANK 2
- #define ICM20948_ACCEL_SMPLRT_DIV_2_BANK 2
- #define ICM20948_ACCEL_CONFIG_1_BANK 2
- #define ICM20948_I2C_MST_CTRL_ADDR 0x01

- #define ICM20948_I2C_SLV0_ADDR_ADDR 0x03
- #define ICM20948_I2C_SLV0_REG_ADDR 0x04
- #define ICM20948_I2C_SLV0_CTRL_ADDR 0x05
- #define ICM20948_I2C_SLV4_ADDR_ADDR 0x13
- #define ICM20948_I2C_SLV4_REG_ADDR 0x14
- #define ICM20948_I2C_SLV4_CTRL_ADDR 0x15
- #define ICM20948_I2C_SLV4_DO_ADDR 0x16
- #define ICM20948_I2C_SLV4_DI_ADDR 0x17
- #define ICM20948_I2C_MST_CTRL_BANK 3
- #define ICM20948_I2C_SLV0_ADDR_BANK 3
- #define ICM20948_I2C_SLV0_REG_BANK 3
- #define ICM20948_I2C_SLV0_CTRL_BANK 3
- #define ICM20948_I2C_SLV4_ADDR_BANK 3
- #define ICM20948_I2C_SLV4_REG_BANK 3
- #define ICM20948_I2C_SLV4_CTRL_BANK 3
- #define ICM20948_I2C_SLV4_DO_BANK 3
- #define ICM20948_I2C_SLV4_DI_BANK 3
- #define AK09916_CNTL2_ADDR 0x31
- #define ICM20948_BANK0_WHO_AM_I_VALUE 0xEA
- #define ICM20948_INT_STATUS_BANK 0
- #define ICM20948_INT_STATUS_1_BANK 0
- #define ICM20948_INT_STATUS_ADDR 0x19
- #define ICM20948_INT_STATUS_1_ADDR 0x1A

Typedefs

- typedef struct icm20948::accel_settings icm20948::accel_settings
- typedef struct icm20948::gyro_settings icm20948::gyro_settings
- typedef struct icm20948::magn_settings icm20948::magn_settings
- typedef struct icm20948::settings icm20948::settings

Enumerations

- enum icm20948::accel_scale { icm20948::ACCEL_2G = 0 , icm20948::ACCEL_4G , icm20948::ACCEL_8G , icm20948::ACCEL_16G }
- enum icm20948::accel_dlpf_config { icm20948::ACCEL_DLPF_246HZ = 0 , icm20948::ACCEL_DLPF_246HZ_2 , icm20948::ACCEL_DLPF_111_4HZ , icm20948::ACCEL_DLPF_50_4HZ , icm20948::ACCEL_DLPF_23_9HZ , icm20948::ACCEL_DLPF_11_5HZ , icm20948::ACCEL_DLPF_5_7HZ , icm20948::ACCEL_DLPF_473HZ }
- enum icm20948::gyro_scale { icm20948::GYRO_250DPS = 0 , icm20948::GYRO_500DPS , icm20948::GYRO_1000DPS , icm20948::GYRO_2000DPS }
- enum icm20948::gyro_dlpf_config { icm20948::GYRO_DLPF_196_6HZ = 0 , icm20948::GYRO_DLPF_151_8HZ , icm20948::GYRO_DLPF_119_5HZ , icm20948::GYRO_DLPF_51_2HZ , icm20948::GYRO_DLPF_23_9HZ , icm20948::GYRO_DLPF_11_6HZ , icm20948::GYRO_DLPF_5_7HZ , icm20948::GYRO_DLPF_361_4HZ }
- enum icm20948::magn_mode { icm20948::MAGN_SHUTDOWN = 0 , icm20948::MAGN_SINGLE = 1 , icm20948::MAGN_10HZ = 2 , icm20948::MAGN_20HZ = 4 , icm20948::MAGN_50HZ = 6 , icm20948::MAGN_100HZ = 8 , icm20948::MAGN_SELF_TEST = 16 }

Functions

- float icm20948::accel_scale_factor (accel_scale scale)
- std::string icm20948::accel_scale_to_str (accel_scale scale)
- std::string icm20948::accel_dlpf_config_to_str (accel_dlpf_config config)
- float icm20948::gyro_scale_factor (gyro_scale scale)
- std::string icm20948::gyro_scale_to_str (gyro_scale scale)
- std::string icm20948::gyro_dlpf_config_to_str (gyro_dlpf_config config)
- std::string icm20948::magn_mode_to_str (magn_mode mode)

8.5.1 Macro Definition Documentation

8.5.1.1 AK09916_CNTL2_ADDR

```
#define AK09916_CNTL2_ADDR 0x31
```

8.5.1.2 ICM20948_ACCEL_CONFIG_1_ADDR

```
#define ICM20948_ACCEL_CONFIG_1_ADDR 0x14
```

8.5.1.3 ICM20948_ACCEL_CONFIG_1_BANK

```
#define ICM20948_ACCEL_CONFIG_1_BANK 2
```

8.5.1.4 ICM20948_ACCEL_OUT_BANK

```
#define ICM20948_ACCEL_OUT_BANK 0
```

8.5.1.5 ICM20948_ACCEL_SMPLRT_DIV_1_ADDR

```
#define ICM20948_ACCEL_SMPLRT_DIV_1_ADDR 0x10
```

8.5.1.6 ICM20948_ACCEL_SMPLRT_DIV_1_BANK

```
#define ICM20948_ACCEL_SMPLRT_DIV_1_BANK 2
```

8.5.1.7 ICM20948_ACCEL_SMPLRT_DIV_2_ADDR

```
#define ICM20948_ACCEL_SMPLRT_DIV_2_ADDR 0x11
```

8.5.1.8 ICM20948_ACCEL_SMPLRT_DIV_2_BANK

```
#define ICM20948_ACCEL_SMPLRT_DIV_2_BANK 2
```

8.5.1.9 ICM20948_ACCEL_XOUT_H_ADDR

```
#define ICM20948_ACCEL_XOUT_H_ADDR 0x2D
```

8.5.1.10 ICM20948_ACCEL_XOUT_L_ADDR

```
#define ICM20948_ACCEL_XOUT_L_ADDR 0x2E
```

8.5.1.11 ICM20948_ACCEL_YOUT_H_ADDR

```
#define ICM20948_ACCEL_YOUT_H_ADDR 0x2F
```

8.5.1.12 ICM20948_ACCEL_YOUT_L_ADDR

```
#define ICM20948_ACCEL_YOUT_L_ADDR 0x30
```

8.5.1.13 ICM20948_ACCEL_ZOUT_H_ADDR

```
#define ICM20948_ACCEL_ZOUT_H_ADDR 0x31
```

8.5.1.14 ICM20948_ACCEL_ZOUT_L_ADDR

```
#define ICM20948_ACCEL_ZOUT_L_ADDR 0x32
```

8.5.1.15 ICM20948_BANK0_WHO_AM_I_VALUE

```
#define ICM20948_BANK0_WHO_AM_I_VALUE 0xEA
```

8.5.1.16 ICM20948_EXT_SLV_SENS_DATA_00_ADDR

```
#define ICM20948_EXT_SLV_SENS_DATA_00_ADDR 0x3B
```

8.5.1.17 ICM20948_EXT_SLV_SENS_DATA_00_BANK

```
#define ICM20948_EXT_SLV_SENS_DATA_00_BANK 0
```

8.5.1.18 ICM20948_GYRO_CONFIG_1_ADDR

```
#define ICM20948_GYRO_CONFIG_1_ADDR 0x01
```

8.5.1.19 ICM20948_GYRO_CONFIG_1_BANK

```
#define ICM20948_GYRO_CONFIG_1_BANK 2
```

8.5.1.20 ICM20948_GYRO_OUT_BANK

```
#define ICM20948_GYRO_OUT_BANK 0
```

8.5.1.21 ICM20948_GYRO_SMPLRT_DIV_ADDR

```
#define ICM20948_GYRO_SMPLRT_DIV_ADDR 0x00
```

8.5.1.22 ICM20948_GYRO_SMPLRT_DIV_BANK

```
#define ICM20948_GYRO_SMPLRT_DIV_BANK 2
```

8.5.1.23 ICM20948_GYRO_XOUT_H_ADDR

```
#define ICM20948_GYRO_XOUT_H_ADDR 0x33
```

8.5.1.24 ICM20948_GYRO_XOUT_L_ADDR

```
#define ICM20948_GYRO_XOUT_L_ADDR 0x34
```

8.5.1.25 ICM20948_GYRO_YOUT_H_ADDR

```
#define ICM20948_GYRO_YOUT_H_ADDR 0x35
```

8.5.1.26 ICM20948_GYRO_YOUT_L_ADDR

```
#define ICM20948_GYRO_YOUT_L_ADDR 0x36
```

8.5.1.27 ICM20948_GYRO_ZOUT_H_ADDR

```
#define ICM20948_GYRO_ZOUT_H_ADDR 0x37
```

8.5.1.28 ICM20948_GYRO_ZOUT_L_ADDR

```
#define ICM20948_GYRO_ZOUT_L_ADDR 0x38
```

8.5.1.29 ICM20948_I2C_ADDR

```
#define ICM20948_I2C_ADDR 0x69
```

8.5.1.30 ICM20948_I2C_MST_CTRL_ADDR

```
#define ICM20948_I2C_MST_CTRL_ADDR 0x01
```

8.5.1.31 ICM20948_I2C_MST_CTRL_BANK

```
#define ICM20948_I2C_MST_CTRL_BANK 3
```

8.5.1.32 ICM20948_I2C_MST_STATUS_ADDR

```
#define ICM20948_I2C_MST_STATUS_ADDR 0x17
```

8.5.1.33 ICM20948_I2C_MST_STATUS_BANK

```
#define ICM20948_I2C_MST_STATUS_BANK 0
```

8.5.1.34 ICM20948_I2C_SLV0_ADDR_ADDR

```
#define ICM20948_I2C_SLV0_ADDR_ADDR 0x03
```

8.5.1.35 ICM20948_I2C_SLV0_ADDR_BANK

```
#define ICM20948_I2C_SLV0_ADDR_BANK 3
```

8.5.1.36 ICM20948_I2C_SLV0_CTRL_ADDR

```
#define ICM20948_I2C_SLV0_CTRL_ADDR 0x05
```

8.5.1.37 ICM20948_I2C_SLV0_CTRL_BANK

```
#define ICM20948_I2C_SLV0_CTRL_BANK 3
```

8.5.1.38 ICM20948_I2C_SLV0_REG_ADDR

```
#define ICM20948_I2C_SLV0_REG_ADDR 0x04
```

8.5.1.39 ICM20948_I2C_SLV0_REG_BANK

```
#define ICM20948_I2C_SLV0_REG_BANK 3
```

8.5.1.40 ICM20948_I2C_SLV4_ADDR_ADDR

```
#define ICM20948_I2C_SLV4_ADDR_ADDR 0x13
```

8.5.1.41 ICM20948_I2C_SLV4_ADDR_BANK

```
#define ICM20948_I2C_SLV4_ADDR_BANK 3
```

8.5.1.42 ICM20948_I2C_SLV4_CTRL_ADDR

```
#define ICM20948_I2C_SLV4_CTRL_ADDR 0x15
```

8.5.1.43 ICM20948_I2C_SLV4_CTRL_BANK

```
#define ICM20948_I2C_SLV4_CTRL_BANK 3
```

8.5.1.44 ICM20948_I2C_SLV4_DI_ADDR

```
#define ICM20948_I2C_SLV4_DI_ADDR 0x17
```

8.5.1.45 ICM20948_I2C_SLV4_DI_BANK

```
#define ICM20948_I2C_SLV4_DI_BANK 3
```

8.5.1.46 ICM20948_I2C_SLV4_DO_ADDR

```
#define ICM20948_I2C_SLV4_DO_ADDR 0x16
```

8.5.1.47 ICM20948_I2C_SLV4_DO_BANK

```
#define ICM20948_I2C_SLV4_DO_BANK 3
```

8.5.1.48 ICM20948_I2C_SLV4_REG_ADDR

```
#define ICM20948_I2C_SLV4_REG_ADDR 0x14
```

8.5.1.49 ICM20948_I2C_SLV4_REG_BANK

```
#define ICM20948_I2C_SLV4_REG_BANK 3
```

8.5.1.50 ICM20948_INT_ENABLE_1_ADDR

```
#define ICM20948_INT_ENABLE_1_ADDR 0x11
```

8.5.1.51 ICM20948_INT_ENABLE_1_BANK

```
#define ICM20948_INT_ENABLE_1_BANK 0
```

8.5.1.52 ICM20948_INT_PIN_CFG_ADDR

```
#define ICM20948_INT_PIN_CFG_ADDR 0x0F
```

8.5.1.53 ICM20948_INT_PIN_CFG_BANK

```
#define ICM20948_INT_PIN_CFG_BANK 0
```

8.5.1.54 ICM20948_INT_STATUS_1_ADDR

```
#define ICM20948_INT_STATUS_1_ADDR 0x1A
```

8.5.1.55 ICM20948_INT_STATUS_1_BANK

```
#define ICM20948_INT_STATUS_1_BANK 0
```

8.5.1.56 ICM20948_INT_STATUS_ADDR

```
#define ICM20948_INT_STATUS_ADDR 0x19
```

8.5.1.57 ICM20948_INT_STATUS_BANK

```
#define ICM20948_INT_STATUS_BANK 0
```

8.5.1.58 ICM20948_MAGN_I2C_ADDR

```
#define ICM20948_MAGN_I2C_ADDR 0x0C
```

8.5.1.59 ICM20948_PWR_MGMT_1_ADDR

```
#define ICM20948_PWR_MGMT_1_ADDR 0x06
```

8.5.1.60 ICM20948_PWR_MGMT_1_BANK

```
#define ICM20948_PWR_MGMT_1_BANK 0
```

8.5.1.61 ICM20948_REG_BANK_SEL_ADDR

```
#define ICM20948_REG_BANK_SEL_ADDR 0x7F
```

8.5.1.62 ICM20948_REG_BANK_SEL_BANK0_VALUE

```
#define ICM20948_REG_BANK_SEL_BANK0_VALUE 0x00
```

8.5.1.63 ICM20948_REG_BANK_SEL_BANK1_VALUE

```
#define ICM20948_REG_BANK_SEL_BANK1_VALUE 0x10
```

8.5.1.64 ICM20948_REG_BANK_SEL_BANK2_VALUE

```
#define ICM20948_REG_BANK_SEL_BANK2_VALUE 0x20
```

8.5.1.65 ICM20948_REG_BANK_SEL_BANK3_VALUE

```
#define ICM20948_REG_BANK_SEL_BANK3_VALUE 0x30
```

8.5.1.66 ICM20948_TEMP_OUT_H_ADDR

```
#define ICM20948_TEMP_OUT_H_ADDR 0x39
```

8.5.1.67 ICM20948_TEMP_OUT_L_ADDR

```
#define ICM20948_TEMP_OUT_L_ADDR 0x3A
```

8.5.1.68 ICM20948_USER_CTRL_ADDR

```
#define ICM20948_USER_CTRL_ADDR 0x03
```

8.5.1.69 ICM20948_USER_CTRL_BANK

```
#define ICM20948_USER_CTRL_BANK 0
```

8.5.1.70 ICM20948_WHO_AM_I_ADDR

```
#define ICM20948_WHO_AM_I_ADDR 0x00
```

8.5.1.71 ICM20948_WHO_AM_I_BANK

```
#define ICM20948_WHO_AM_I_BANK 0
```

8.6 Icm20948driver.hpp

Go to the documentation of this file.

```
00001 // This header defines the register map, configuration types, and public driver
00002 // interface for the ICM-20948 IMU and its onboard magnetometer.
00003 //
00004 // SOLID principles:
00005 //   S - This file centralises device-specific constants, configuration structures,
00006 //       and the public interface for the IMU driver in one place.
00007 //   O - Configuration is represented through settings types, allowing the driver to
00008 //       be reconfigured without changing client code that uses the interface.
00009 //   I - Configuration is grouped into smaller settings types (accelerometer,
00010 //       gyroscope, magnetometer) so callers interact with the parts of the
00011 //       device configuration they actually need
00012
00013
00014 #pragma once
00015
00016 #include <cstdint>
00017 #include <string>
00018 #include <linux/i2c-dev.h>
00019 #include <linux/i2c.h>
00020 #include "yaml-cpp/yaml.h"
00021
00022
00023 // Register addresses and constants for the ICM-20948 IMU and onboard magnetometer.
00024 //  Register map
00025
00026 #define ICM20948_I2C_ADDR       0x69
00027 #define ICM20948_MAGN_I2C_ADDR  0x0C
00028
00029 // Bank-select register and encoded bank values.
00030 // These constants isolate device-specific magic numbers from the driver logic.
00031 #define ICM20948_REG_BANK_SEL_ADDR          0x7F
00032 #define ICM20948_REG_BANK_SEL_BANK0_VALUE   0x00
00033 #define ICM20948_REG_BANK_SEL_BANK1_VALUE   0x10
00034 #define ICM20948_REG_BANK_SEL_BANK2_VALUE   0x20
00035 #define ICM20948_REG_BANK_SEL_BANK3_VALUE   0x30
00036
00037 // Bank 0
00038 // Output and control registers used for identity, power, interrupts,
00039 // and reading decoded sensor data.
00040 #define ICM20948_WHO_AM_I_ADDR              0x00
00041 #define ICM20948_USER_CTRL_ADDR             0x03
00042 #define ICM20948_PWR_MGMT_1_ADDR            0x06
00043 #define ICM20948_INT_PIN_CFG_ADDR           0x0F
00044 #define ICM20948_INT_ENABLE_1_ADDR          0x11
00045 #define ICM20948_I2C_MST_STATUS_ADDR        0x17
00046 #define ICM20948_ACCEL_XOUT_H_ADDR          0x2D
00047 #define ICM20948_ACCEL_XOUT_L_ADDR          0x2E
00048 #define ICM20948_ACCEL_YOUT_H_ADDR          0x2F
00049 #define ICM20948_ACCEL_YOUT_L_ADDR          0x30
00050 #define ICM20948_ACCEL_ZOUT_H_ADDR          0x31
00051 #define ICM20948_ACCEL_ZOUT_L_ADDR          0x32
00052 #define ICM20948_GYRO_XOUT_H_ADDR           0x33
00053 #define ICM20948_GYRO_XOUT_L_ADDR           0x34
00054 #define ICM20948_GYRO_YOUT_H_ADDR           0x35
00055 #define ICM20948_GYRO_YOUT_L_ADDR           0x36
```

```
#define ICM20948_GYRO_ZOUT_H_ADDR           0x37
#define ICM20948_GYRO_ZOUT_L_ADDR           0x38
#define ICM20948_TEMP_OUT_H_ADDR            0x39
#define ICM20948_TEMP_OUT_L_ADDR            0x3A
#define ICM20948_EXT_SLV_SENS_DATA_00_ADDR  0x3B

// Bank IDs for the corresponding registers above.
// Keeping bank IDs alongside register definitions reduces the chance of
// mismatched bank/register access in the implementation.
#define ICM20948_WHO_AM_I_BANK              0
#define ICM20948_USER_CTRL_BANK             0
#define ICM20948_PWR_MGMT_1_BANK            0
#define ICM20948_INT_PIN_CFG_BANK           0
#define ICM20948_INT_ENABLE_1_BANK          0
#define ICM20948_I2C_MST_STATUS_BANK        0
#define ICM20948_ACCEL_OUT_BANK             0
#define ICM20948_GYRO_OUT_BANK              0
#define ICM20948_EXT_SLV_SENS_DATA_00_BANK  0

// Bank 2
// Configuration registers for sample rate, scale, and digital filtering.
#define ICM20948_GYRO_SMPLRT_DIV_ADDR       0x00
#define ICM20948_GYRO_CONFIG_1_ADDR         0x01
#define ICM20948_ACCEL_SMPLRT_DIV_1_ADDR    0x10
#define ICM20948_ACCEL_SMPLRT_DIV_2_ADDR    0x11
#define ICM20948_ACCEL_CONFIG_1_ADDR        0x14

// Bank IDs for accel/gyro configuration registers.
#define ICM20948_GYRO_SMPLRT_DIV_BANK       2
#define ICM20948_GYRO_CONFIG_1_BANK         2
#define ICM20948_ACCEL_SMPLRT_DIV_1_BANK    2
#define ICM20948_ACCEL_SMPLRT_DIV_2_BANK    2
#define ICM20948_ACCEL_CONFIG_1_BANK        2

// Bank 3
// Registers for the internal I2C master used to communicate with the
// onboard magnetometer through the main IMU device.
#define ICM20948_I2C_MST_CTRL_ADDR          0x01
#define ICM20948_I2C_SLV0_ADDR_ADDR         0x03
#define ICM20948_I2C_SLV0_REG_ADDR          0x04
#define ICM20948_I2C_SLV0_CTRL_ADDR         0x05
#define ICM20948_I2C_SLV4_ADDR_ADDR         0x13
#define ICM20948_I2C_SLV4_REG_ADDR          0x14
#define ICM20948_I2C_SLV4_CTRL_ADDR         0x15
#define ICM20948_I2C_SLV4_DO_ADDR           0x16
#define ICM20948_I2C_SLV4_DI_ADDR           0x17

// Bank IDs for the internal I2C master interface.
#define ICM20948_I2C_MST_CTRL_BANK          3
#define ICM20948_I2C_SLV0_ADDR_BANK         3
#define ICM20948_I2C_SLV0_REG_BANK          3
#define ICM20948_I2C_SLV0_CTRL_BANK         3
#define ICM20948_I2C_SLV4_ADDR_BANK         3
#define ICM20948_I2C_SLV4_REG_BANK          3
#define ICM20948_I2C_SLV4_CTRL_BANK         3
#define ICM20948_I2C_SLV4_DO_BANK           3
#define ICM20948_I2C_SLV4_DI_BANK           3

// AK09916 magnetometer control register and expected WHO_AM_I value.
#define AK09916_CNTL2_ADDR                  0x31
#define ICM20948_BANK0_WHO_AM_I_VALUE       0xEA


// Interrupt status registers.
// These support event-driven sampling by allowing higher-level code to
// check whether new data is ready before attempting a read.
#define ICM20948_INT_STATUS_BANK            0
#define ICM20948_INT_STATUS_1_BANK          0

#define ICM20948_INT_STATUS_ADDR            0x19
#define ICM20948_INT_STATUS_1_ADDR          0x1A


//  Settings types

namespace icm20948
{
    // Accelerometer full-scale range.
    // Using an enum keeps configuration values readable and avoids scattering
    // raw register meanings through application code.
    typedef enum {
        ACCEL_2G  = 0,
        ACCEL_4G,
        ACCEL_8G,
        ACCEL_16G
    } accel_scale;

```

```
    // Accelerometer digital low-pass filter config.
    typedef enum {
        ACCEL_DLPF_246HZ   = 0,
        ACCEL_DLPF_246HZ_2,
        ACCEL_DLPF_111_4HZ,
        ACCEL_DLPF_50_4HZ,
        ACCEL_DLPF_23_9HZ,
        ACCEL_DLPF_11_5HZ,
        ACCEL_DLPF_5_7HZ,
        ACCEL_DLPF_473HZ
    } accel_dlpf_config;

    // Accelerometer runtime configuration.
    // Grouping these parameters in one struct improves clarity and keeps the
    // public interface cleaner than passing many separate constructor arguments.
    typedef struct accel_settings {
        uint16_t          sample_rate_div;
        accel_scale       scale;
        bool              dlpf_enable;
        accel_dlpf_config dlpf_config;

        accel_settings(uint16_t          sample_rate_div = 0,
                       accel_scale       scale           = ACCEL_2G,
                       bool              dlpf_enable     = true,
                       accel_dlpf_config dlpf_config     = ACCEL_DLPF_246HZ)
            : sample_rate_div(sample_rate_div), scale(scale),
              dlpf_enable(dlpf_enable), dlpf_config(dlpf_config) {}
    } accel_settings;

    // Gyroscope full-scale range.
    typedef enum {
        GYRO_250DPS  = 0,
        GYRO_500DPS,
        GYRO_1000DPS,
        GYRO_2000DPS
    } gyro_scale;

    // Gyroscope digital low-pass filter config.
    typedef enum {
        GYRO_DLPF_196_6HZ = 0,
        GYRO_DLPF_151_8HZ,
        GYRO_DLPF_119_5HZ,
        GYRO_DLPF_51_2HZ,
        GYRO_DLPF_23_9HZ,
        GYRO_DLPF_11_6HZ,
        GYRO_DLPF_5_7HZ,
        GYRO_DLPF_361_4HZ
    } gyro_dlpf_config;

    // Gyroscope runtime configuration.
    typedef struct gyro_settings {
        uint8_t          sample_rate_div;
        gyro_scale       scale;
        bool             dlpf_enable;
        gyro_dlpf_config dlpf_config;

        gyro_settings(uint8_t          sample_rate_div = 0,
                      gyro_scale       scale           = GYRO_250DPS,
                      bool             dlpf_enable     = true,
                      gyro_dlpf_config dlpf_config     = GYRO_DLPF_196_6HZ)
            : sample_rate_div(sample_rate_div), scale(scale),
              dlpf_enable(dlpf_enable), dlpf_config(dlpf_config) {}
    } gyro_settings;

    // Magnetometer operating mode.
    typedef enum {
        MAGN_SHUTDOWN  = 0,
        MAGN_SINGLE    = 1,
        MAGN_10HZ      = 2,
        MAGN_20HZ      = 4,
        MAGN_50HZ      = 6,
        MAGN_100HZ     = 8,
        MAGN_SELF_TEST = 16
    } magn_mode;

    // Magnetometer runtime configuration.
    typedef struct magn_settings {
        magn_mode mode;
        explicit magn_settings(magn_mode mode = MAGN_100HZ) : mode(mode) {}
    } magn_settings;

    // Aggregate device configuration.
    // This gives client code a single configuration object, which improves
    // readability and makes the driver easier to construct and reconfigure.
    typedef struct settings {
        accel_settings accel;
        gyro_settings  gyro;
```

```
        magn_settings  magn;

        settings(accel_settings accel = accel_settings(),
                 gyro_settings  gyro  = gyro_settings(),
                 magn_settings  magn  = magn_settings())
            : accel(accel), gyro(gyro), magn(magn) {}

        // Construct settings from a YAML config node.
        // This supports configuration-driven setup without hard-coding all
        // parameters in application code.
        explicit settings(YAML::Node config_file_node);
    } settings;

    // Helpers for converting config enums to scale factors / readable strings.
    // These utility functions separate presentation and conversion logic from
    // the main driver code, improving maintainability.
    float       accel_scale_factor(accel_scale scale);
    std::string accel_scale_to_str(accel_scale scale);
    std::string accel_dlpf_config_to_str(accel_dlpf_config config);

    float       gyro_scale_factor(gyro_scale scale);
    std::string gyro_scale_to_str(gyro_scale scale);
    std::string gyro_dlpf_config_to_str(gyro_dlpf_config config);

    std::string magn_mode_to_str(magn_mode mode);

    // One fully decoded IMU sample.
    // This struct provides a clean data-transfer object so callers can work
    // with physical units rather than raw register bytes.
    struct IMUSample {
        float ax, ay, az;
        float gx, gy, gz;
        float mx, my, mz;
    };

    // ICM20948_I2C encapsulates the device driver for the IMU over Linux I2C.
    //
    // SOLID design intention:
    // - Single Responsibility:
    //   This class focuses on IMU communication, configuration, and conversion
    //   of raw register data into decoded sensor values.
    // - Encapsulation:
    //   Register access, bank switching, and magnetometer control are hidden
    //   behind a higher-level public API.
    // - Open/Closed:
    //   Client code interacts through configuration objects and high-level
    //   methods, allowing internal implementation details to evolve without
    //   forcing changes on users of the class.
    //
    // Note:
    // This driver necessarily owns several low-level responsibilities because
    // it acts as the hardware abstraction boundary for the IMU device.
    class ICM20948_I2C
    {
    public:
        // Most recent decoded sensor values.
        // These cached arrays provide direct access to the latest successful
        // reading while keeping raw register handling internal to the class.
        float accel[3];
        float gyro[3];
        float magn[3];

        // Active driver settings.
        // Storing settings here keeps configuration tied to the device instance.
        icm20948::settings settings;

        ICM20948_I2C(unsigned           i2c_bus,
                     unsigned           i2c_address = ICM20948_I2C_ADDR,
                     icm20948::settings settings    = icm20948::settings());

        // Public API for device lifecycle and sensor access.
        // Higher-level code interacts through these methods rather than
        // directly manipulating device registers.
        bool init();
        bool reset();
        bool wake();
        bool set_settings();
        bool read_accel_gyro();
        bool read_magn();
        bool read_sample(IMUSample& sample);
        bool check_DRDY_INT();

    private:
        // Linux I2C file descriptor and device identity.
        // These implementation details are kept private so the transport layer
        // remains hidden from the rest of the application.
        int       _i2c_fd;
```

```
        unsigned  _i2c_bus;
        unsigned  _i2c_address;

        // Cached register bank to avoid redundant bank switches.
        // This is a performance and consistency optimisation internal to the driver.
        uint8_t   _current_bank;

        // Cached conversion factors from raw counts to physical units.
        // Keeping these values private prevents duplication of conversion logic.
        float     _accel_scale_factor;
        float     _gyro_scale_factor;
        float     _magn_scale_factor;

        // Device setup helpers.
        // These methods isolate configuration subtasks, which keeps the public
        // interface small and the implementation more maintainable.
        bool _enable_data_ready_interrupt();
        bool _set_bank(uint8_t bank);
        bool _set_accel_sample_rate_div();
        bool _set_accel_range_dlpf();
        bool _set_gyro_sample_rate_div();
        bool _set_gyro_range_dlpf();

        // Magnetometer setup via the ICM-20948 internal I2C master.
        // This complexity is hidden from callers so they can treat the driver
        // as a single IMU abstraction.
        bool _magnetometer_init();
        bool _magnetometer_enable();
        bool _magnetometer_set_mode();
        bool _magnetometer_configured();
        bool _magnetometer_set_readout();
        bool _chip_i2c_master_reset();

        // Low-level register access helpers.
        // These centralise raw byte/bit operations so direct register handling
        // is not scattered across the driver implementation.
        bool _write_byte(uint8_t bank, uint8_t reg, uint8_t byte);
        bool _read_byte(uint8_t bank, uint8_t reg, uint8_t& byte);
        bool _write_bit(uint8_t bank, uint8_t reg, uint8_t bit_pos, bool bit);
        bool _read_bit(uint8_t bank, uint8_t reg, uint8_t bit_pos, bool& bit);
        bool _read_block_bytes(uint8_t bank, uint8_t start_reg, uint8_t* bytes, int length);
        bool _write_mag_byte(uint8_t mag_reg, uint8_t byte);
        bool _read_mag_byte(uint8_t mag_reg, uint8_t& byte);


    };

} // namespace icm20948
```

8.7 src/libs/IMU/Include/imureader.hpp File Reference

```
#include <atomic>
#include <functional>
#include <thread>
#include "Icm20948driver.hpp"
#include <gpiod.h>
#include <future>
```

Include dependency graph for imureader.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class IMUReader

8.8 imureader.hpp

Go to the documentation of this file.

```
// IMUReader provides a threaded, event-driven wrapper around the low-level
// ICM-20948 driver.
```

```

// SOLID principles:
//   S - This class is responsible for coordinating asynchronous IMU sample
//       acquisition from the data-ready GPIO and forwarding decoded samples to
//       client code.
//   O - New behaviours can be attached through the callback interface without
//       changing the reader logic itself.
//   D - The low-level register and I2C interaction remain inside the IMU driver,
//       while this class focuses on threading and sample dispatch.
//
// Threaded IMU reader driven by the data-ready GPIO.

#pragma once

#include <atomic>
#include <functional>
#include <thread>

#include "Icm20948driver.hpp"
#include <gpiod.h>
#include <future>

class IMUReader
{
public:
    // Callback invoked for each decoded IMU sample.
    // Using a callback keeps the class open for extension: logging, filtering,
    // fusion, motion detection, or UI updates can be added without modifying
    // the reader implementation.
    using SampleCallback = std::function<void(float ax, float ay, float az,
                                              float gx, float gy, float gz,
                                              float mx, float my)>;

    // Construct the reader with its hardware configuration and runtime settings.
    // The IMU configuration is injected from outside so the reader can be
    // reused with different buses, addresses, and sensor settings.
    IMUReader(unsigned           i2c_bus,
              unsigned           i2c_address = ICM20948_I2C_ADDR,
              const char*        gpiochip    = "/dev/gpiochip0",
              unsigned           rdy_line    = 27,
              icm20948::settings settings    = icm20948::settings());

    // Destructor ensures the threaded reader can clean up safely through the
    // class interface.
    ~IMUReader();

    // Initialise the underlying IMU device before starting the worker thread.
    bool init();

    // Register client code to receive new IMU samples.
    // This keeps sample processing outside the reader itself, improving
    // flexibility and separation of concerns.
    void setCallback(SampleCallback cb);

    // Start the background reader thread and GPIO-driven event loop.
    bool start();

    // Stop the worker thread and release runtime resources cleanly.
    void stop();
    icm20948::ICM20948_I2C& getIMU();
private:
    // Background worker loop waiting for DRDY events and dispatching samples.
    // This keeps blocking/event-loop behaviour internal to the class so client
    // code does not need to manage low-level synchronisation details.
    void worker();

    // Low-level IMU driver instance.
    // Hardware-specific register access is delegated to the driver so this
    // class can focus on threaded sample acquisition and dispatch.
    icm20948::ICM20948_I2C imu_;

    // Client callback for newly acquired samples.
    // Stored privately so callback ownership and invocation remain controlled
    // by the reader.
    SampleCallback         callback_;

    // Background reader thread.
    // Thread lifetime is encapsulated inside the class rather than exposed to
    // higher-level application code.
    std::thread            worker_thread_;

    // Run flag shared with the worker thread.
    // Atomic state allows safe coordination between start/stop and the worker.
    std::atomic_bool       running_;

    // GPIO chip path and DRDY line number.
    // These hardware configuration details are stored with the reader instance
```

```
    // so callers only configure them once during construction.
    const char*         gpiochip_path_;
    unsigned            rdy_line_;

    // libgpiod handles for the DRDY line.
    // These low-level resources are kept private to preserve encapsulation.
    gpiod_chip*         chip_;
    gpiod_line_request* request_;

    // File descriptors used by the event loop.
    // Internal event-loop state is hidden so callers interact only through the
    // public start/stop interface.
    int                 gpio_fd_;
    int                 stop_fd_;
};
```

8.9 src/libs/IMU_math/Include/positiondetector.hpp File Reference

```
#include <cmath>
#include <functional>
```

Include dependency graph for positiondetector.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PositionDetector
- struct PositionDetector::Config

8.10 positiondetector.hpp

Go to the documentation of this file.

```
/*
    This header declares the position detector class.
    It stores the configuration for detecting an upright position,
    processes sensor samples, tracks stability, and uses a callback
    when the required position has been confirmed.
*/

#pragma once

#include <cmath>
#include <functional>

// Declares the position detector class, including its callback type, configuration settings, and main functions for processing samples and resetting state.
class PositionDetector {
public:
    using StateCallback = std::function<void(bool upright)>;

    struct Config {
        float refX =  9.88f;
        float refY = -0.87f;
        float refZ =  0.10f;

        float minG = 7.0f;
        float maxG = 12.5f;
        float minDot = 0.90f;

        float refHeadingDeg = 25.0f;
        float headingToleranceDeg = 180.0f;

        int stabilitySamples = 10;
    };

    PositionDetector();
    explicit PositionDetector(const Config& c);
    virtual ~PositionDetector() = default;

    virtual void onSample(float ax, float ay, float az, float mx, float my);
```

```
    void setCallback(StateCallback cb) { callback = std::move(cb); }
    // Clears stability count and confirmed state so the callback can re-fire
    // after a cycle reset while the device stays at the same position.
    void resetState();

private:
    // Declares the helper function and private members used to store configuration, reference values, stability count, state, and callback data.
    static float wrapAngleDeg(float a);

    Config cfg;
    float nRefX, nRefY, nRefZ;   // normalised ref vector, computed from cfg
    int stableCount;
    bool confirmed;
    StateCallback callback;
};
```

8.11 src/libs/IMU_math/Include/SwingCalibrator.hpp File Reference

`#include <functional>`

Include dependency graph for SwingCalibrator.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class SwingCalibrator

8.12 SwingCalibrator.hpp

Go to the documentation of this file.

```
#pragma once

#include <functional>

class SwingCalibrator {
public:
    using DoneCallback = std::function<void(float bx, float by, float bz, bool initial)>;

    explicit SwingCalibrator(int initial_samples = 200, int recal_samples = 50);

    void setCallback(DoneCallback cb);

    bool feed(float ax, float ay, float az);

    void triggerRecal();

    bool isReady() const;

    float biasX() const { return bias_x_; }
    float biasY() const { return bias_y_; }
    float biasZ() const { return bias_z_; }

private:
    enum class State { INITIAL, READY, RECAL };

    int   initial_samples_;
    int   recal_samples_;
    State state_;
    int   count_;

    float sum_x_, sum_y_, sum_z_;

    float bias_x_, bias_y_, bias_z_;

    DoneCallback callback_;

    void accumulate(float ax, float ay, float az, int target, bool initial);
};
```

8.13 src/libs/IMU_math/Include/SwingDetector.hpp File Reference

```
#include <functional>
```

Include dependency graph for SwingDetector.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class SwingDetector

8.14 SwingDetector.hpp

Go to the documentation of this file.

```
#pragma once

#include <functional>

// Maps linear acceleration magnitude to duty-cycle bands.
// Only emits when the band changes.
// a >= 25 → "Highest Duty Cycle"
// 15 <= a < 25 → "Medium Duty Cycle"
// 5 <= a < 15 → "Low Duty Cycle"
// a < 5 → "No Duty Cycle"

class SwingDetector {
public:
    using EventCallback = std::function<void(const char* level)>;

    SwingDetector();

    void setCallback(EventCallback cb);

    // Process one accel-magnitude sample and emit on band change.
    void detect(float accel_mag);

    // Clear cached state so the next detect() emits unconditionally.
    void reset();

private:
    enum class Level { NONE, LOW, MEDIUM, HIGH };

    // Convert accel magnitude to a duty-cycle band.
    static Level classify(float accel_mag);

    // Convert enum value to callback label.
    static const char* label(Level level);

    // Last emitted band.
    Level last_level_;

    EventCallback callback_;
};
```

8.15 src/libs/IMU_math/Include/SwingProcessor.hpp File Reference

```
#include "SwingCalibrator.hpp"
#include "positiondetector.hpp"
#include <chrono>
#include <functional>
```

Include dependency graph for SwingProcessor.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class SwingProcessor

Variables

- static constexpr unsigned I2C_BUS = 1
- static constexpr int CALIB_SAMPLES = 50
- static constexpr int RECAL_SAMPLES = 20
- static constexpr float ACCEL_DEADBAND = 1.0f

8.15.1 Variable Documentation

8.15.1.1 ACCEL_DEADBAND

```
constexpr float ACCEL_DEADBAND = 1.0f  [static], [constexpr]
```

8.15.1.2 CALIB_SAMPLES

```
constexpr int CALIB_SAMPLES = 50  [static], [constexpr]
```

8.15.1.3 I2C_BUS

```
constexpr unsigned I2C_BUS = 1  [static], [constexpr]
```

8.15.1.4 RECAL_SAMPLES

```
constexpr int RECAL_SAMPLES = 20  [static], [constexpr]
```

8.16 SwingProcessor.hpp

Go to the documentation of this file.

```
/*
    This header declares the swing processor class.
    It combines calibration, position detection, force input,
    and sample processing to control when swing feedback becomes active.
*/

#pragma once

#include "SwingCalibrator.hpp"
#include "positiondetector.hpp"

#include <chrono>
#include <functional>

// Defines the I2C bus and the main constants used for calibration, recalibration, and acceleration deadband control.
static constexpr unsigned I2C_BUS        = 1;
static constexpr int      CALIB_SAMPLES  = 50;
static constexpr int      RECAL_SAMPLES  = 20;
static constexpr float    ACCEL_DEADBAND = 1.0f;

/*
```

```
    Gate logic:
   1. When the correct start position is held, isArmed_ becomes true and the bias is recalibrated.
      The system must still be in this valid position when the piezo is triggered.
   2. When the piezo is pressed while armed, isActive_ becomes true and the motor feedback starts.
      Once active, any further piezo events are ignored to avoid false triggers caused by motor vibration.
   3. When the SwingFeedback timer ends, reset() is called to clear both state flags.
*/

// Declares the swing processor class, including its callback types and main functions for handling force, position, reset, and sensor sample processing.
class SwingProcessor : public PositionDetector {
public:
    using MagnitudeCallback     = std::function<void(float accel_mag)>;
    using PositionStateCallback = std::function<void(bool upright)>;

    explicit SwingProcessor(float accel_deadband = ACCEL_DEADBAND,
                            int   calib_samples  = CALIB_SAMPLES,
                            int   recal_samples  = RECAL_SAMPLES);

    void setMagnitudeCallback(MagnitudeCallback cb);
    void setPositionCallback(PositionStateCallback cb);

    void onForceReady(bool correctForce);
    bool isActive() const { return isActive_; }
    void onPositionReady(bool upright);
    void reset();

    void onSample(float ax, float ay, float az,
                  float mx, float my) override;

    void operator()(float ax, float ay, float az,
                    float gx, float gy, float gz,
                    float mx, float my);

private:
    // Declares the private members used to store thresholds, calibration, callbacks, activity state, and reset timing.
    float                 accel_deadband_;
    SwingCalibrator       calibrator_;
    MagnitudeCallback     mag_callback_;
    PositionStateCallback pos_callback_;

    bool isArmed_  = false;   // position confirmed, ready for piezo
    bool isActive_ = false;   // gate open, motor feedback running

    static constexpr int COOLDOWN_MS = 500;
    std::chrono::steady_clock::time_point reset_time_{};


};
```

8.17 src/libs/IMU_math/positiondetector.cpp File Reference

`#include "positiondetector.hpp"`

Include dependency graph for positiondetector.cpp:

8.18 src/libs/IMU_math/SwingCalibrator.cpp File Reference

`#include "SwingCalibrator.hpp"`

Include dependency graph for SwingCalibrator.cpp:

8.19 src/libs/IMU_math/SwingDetector.cpp File Reference

`#include "SwingDetector.hpp"`

Include dependency graph for SwingDetector.cpp:

8.20 src/libs/IMU_math/SwingProcessor.cpp File Reference

```
#include "SwingProcessor.hpp"
#include <cmath>
```

Include dependency graph for SwingProcessor.cpp:

8.21 src/libs/LEDs/Include/led_controller.h File Reference

```
#include "ledcallback.hpp"
#include <chrono>
```

Include dependency graph for led_controller.h: This graph shows which files directly or indirectly include this file:

Classes

- struct LEDControllerSettings

  This header declares the LED controller settings and class.
- class LEDController

  Declares the LED controller class and its main functions for setting, switching, flashing, and updating the LED output.

8.22 led_controller.h

Go to the documentation of this file.

```
00001
00005
00006
00007 #ifndef LED_CONTROLLER_H
00008 #define LED_CONTROLLER_H
00009
00010 #include "ledcallback.hpp"
00011 #include <chrono>
00012
00014 struct LEDControllerSettings {
00015     int          chipNumber = 0;
00016     unsigned int greenGpio  = 25;
00017 };
00018
00020 class LEDController : public SimpleLEDController
00021 {
00022 public:
00023     LEDController(LEDControllerSettings settings = LEDControllerSettings());
00024     ~LEDController() override;
00025
00026     void set(bool on) override;
00027
00028     void greenOn();
00029     void greenOff();
00030     void allOff();
00031
00032     void flashGreen(int flashMs);
00033     void service();
00034
00035 private:
00037     void updateOutputs();
00038
00039     bool greenActive_ = false;
00040     std::chrono::steady_clock::time_point greenOffTime_{};
00041 };
00042
00043 #endif
```

8.23 src/libs/LEDs/Include/ledcallback.hpp File Reference

```
#include <gpiod.hpp>
#include <memory>
```

Include dependency graph for ledcallback.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class SimpleLEDController

  This header declares a simple LED controller base class. This is to control the LED associated with the IMU swing position.

8.24 ledcallback.hpp

Go to the documentation of this file.

```
00001
00002
00016
00021
00022
00023 #ifndef LEDCALLBACK_HPP
00024 #define LEDCALLBACK_HPP
00025
00026 #include <gpiod.hpp>
00027 #include <memory>
00028
00032 class SimpleLEDController
00033 {
00034 public:
00038     SimpleLEDController(int pinNo, int chipNo = 0);
00039
00042     virtual ~SimpleLEDController();
00043
00046     virtual void set(bool on);
00047
00051     virtual void flashGreen(int /*flashMs*/) {}
00052
00055     virtual void service()                   {}
00056
00057 protected:
00060     SimpleLEDController() = default;
00061
00064     int pin        = -1;
00065     int chipNumber = -1;
00066
00069     std::shared_ptr<gpiod::chip>         chip;
00070     std::shared_ptr<gpiod::line_request> request;
00071 };
00072
00073 #endif
```

8.25 src/libs/LEDs/led_controller.cpp File Reference

```
#include "led_controller.h"
#include <gpiod.hpp>
#include <stdexcept>
#include <cstdio>
```

Include dependency graph for led_controller.cpp:

8.26 src/libs/LEDs/ledcallback.cpp File Reference

```
#include "ledcallback.hpp"
#include <stdexcept>
#include <string>
```

Include dependency graph for ledcallback.cpp:

8.27 src/libs/Motor/Include/rpi_pwm.h File Reference

```
#include <atomic>
#include <chrono>
#include <gpiod.h>
#include <iostream>
#include <sys/timerfd.h>
#include <thread>
#include <unistd.h>
```

Include dependency graph for rpi_pwm.h: This graph shows which files directly or indirectly include this file:

Classes

- class RPI_PWM

Variables

- static constexpr unsigned int PWM_GPIO_LINE = 18
- static constexpr int PWM_FREQ_HZ = 30

8.27.1 Variable Documentation

8.27.1.1 PWM_FREQ_HZ

```
constexpr int PWM_FREQ_HZ = 30  [static], [constexpr]
```

8.27.1.2 PWM_GPIO_LINE

```
constexpr unsigned int PWM_GPIO_LINE = 18  [static], [constexpr]
```

8.28 rpi_pwm.h

Go to the documentation of this file.

```
/*
    This header declares a simple software PWM class for the Raspberry Pi.
    It controls a GPIO output using a timed loop, allows the duty cycle
    to be updated, and manages the resources needed to run the PWM signal.
*/

#pragma once

#include <atomic>
#include <chrono>
#include <gpiod.h>
#include <iostream>
#include <sys/timerfd.h>
#include <thread>
#include <unistd.h>

// Defines the GPIO line and PWM frequency used by the software PWM output.
static constexpr unsigned int PWM_GPIO_LINE = 18;
static constexpr int PWM_FREQ_HZ = 30;

// Declares the PWM class and its main functions for starting, stopping, and updating the duty cycle.
class RPI_PWM {
public:
    RPI_PWM() : duty_(0.0f), running_(false), chip_(nullptr), request_(nullptr) {}

    bool start() {
        chip_ = gpiod_chip_open("/dev/gpiochip0");
        if (!chip_) return false;

        gpiod_line_settings* settings = gpiod_line_settings_new();
        gpiod_line_settings_set_direction(settings, GPIOD_LINE_DIRECTION_OUTPUT);
        gpiod_line_settings_set_output_value(settings, GPIOD_LINE_VALUE_INACTIVE);

        gpiod_line_config* line_cfg = gpiod_line_config_new();
        gpiod_line_config_add_line_settings(line_cfg, &PWM_GPIO_LINE, 1, settings);

        gpiod_request_config* req_cfg = gpiod_request_config_new();
        gpiod_request_config_set_consumer(req_cfg, "soft_pwm");

        request_ = gpiod_chip_request_lines(chip_, req_cfg, line_cfg);

        gpiod_line_settings_free(settings);
        gpiod_line_config_free(line_cfg);
        gpiod_request_config_free(req_cfg);

        if (!request_) {
            gpiod_chip_close(chip_);
            return false;
        }

        running_ = true;
        thread_ = std::thread(&RPI_PWM::loop, this);
        std::cout « "Software PWM started on GPIO " « PWM_GPIO_LINE « "\n";
        return true;
    }

    void stop() {
        running_ = false;
        if (thread_.joinable()) thread_.join();
        if (request_) {
            gpiod_line_request_set_value(request_, PWM_GPIO_LINE, GPIOD_LINE_VALUE_INACTIVE);
            gpiod_line_request_release(request_);
            request_ = nullptr;
        }
        if (chip_) {
            gpiod_chip_close(chip_);
            chip_ = nullptr;
        }
    }

    void setDutyCycle(float percent) {
        if (percent < 0.0f)   percent = 0.0f;
        if (percent > 100.0f) percent = 100.0f;
        duty_ = percent;
    }

    float getDutyCycle() const { return duty_.load(); }

    ~RPI_PWM() { stop(); }

private:
    // Declares the helper functions and private members used for timing, GPIO control, thread
handling, and PWM state.
```

```
    static itimerspec usToTimerspec(long us) {
        itimerspec ts{};
        ts.it_value.tv_sec     = us / 1000000;
        ts.it_value.tv_nsec    = (us % 1000000) * 1000;
        ts.it_interval.tv_sec  = 0;
        ts.it_interval.tv_nsec = 0;
        return ts;
    }

    void waitUs(int tfd, int us) {
        itimerspec ts = usToTimerspec(us);
        timerfd_settime(tfd, 0, &ts, nullptr);
        uint64_t exp;
        ::read(tfd, &exp, sizeof(exp));
    }

    void loop() {
        const int period_us = 1000000 / PWM_FREQ_HZ;

        int tfd = timerfd_create(CLOCK_MONOTONIC, 0);
        if (tfd < 0) return;

        while (running_) {
            float d    = duty_.load();
            int on_us  = (int)(period_us * d / 100.0f);
            int off_us = period_us - on_us;

            if (d >= 100.0f) {
                gpiod_line_request_set_value(request_, PWM_GPIO_LINE, GPIOD_LINE_VALUE_ACTIVE);
                waitUs(tfd, period_us);
            } else if (d <= 0.0f) {
                gpiod_line_request_set_value(request_, PWM_GPIO_LINE, GPIOD_LINE_VALUE_INACTIVE);
                waitUs(tfd, period_us);
            } else {
                gpiod_line_request_set_value(request_, PWM_GPIO_LINE, GPIOD_LINE_VALUE_ACTIVE);
                waitUs(tfd, on_us);
                gpiod_line_request_set_value(request_, PWM_GPIO_LINE, GPIOD_LINE_VALUE_INACTIVE);
                waitUs(tfd, off_us);
            }
        }

        ::close(tfd);
    }

    std::atomic<float>  duty_;
    std::atomic<bool>   running_;
    std::thread         thread_;
    gpiod_chip*         chip_;
    gpiod_line_request* request_;
};
```

8.29 src/libs/Motor/Include/swingfeedback.h File Reference

```
#include "rpi_pwm.h"
#include <atomic>
#include <chrono>
#include <functional>
```

Include dependency graph for swingfeedback.h: This graph shows which files directly or indirectly include this file:

Classes

- class SwingFeedback

8.30 swingfeedback.h

Go to the documentation of this file.

```
/*
    This header declares the swing feedback class.
    It manages PWM feedback output, handles level changes,
```

```
    checks for time-outs, and supports a reset callback.
*/

#pragma once

#include "rpi_pwm.h"

#include <atomic>
#include <chrono>
#include <functional>

// Declares the swing feedback class, including its callback type and main functions for handling feedback, time-outs, and reset behaviour.
class SwingFeedback {
public:
    using ResetCallback = std::function<void()>;

    explicit SwingFeedback(RPI_PWM& pwm);

    void setResetCallback(ResetCallback cb);
    void checkTimeout();
    void onLevel(const char* level);
    void forceOff();

private:
    // Declares the internal worker function and the private members used to track PWM output, callback state, pending level, and timeout timing.
    void worker();

    RPI_PWM&           pwm_;
    ResetCallback      reset_cb_;
    std::atomic<bool>  busy_{false};
    std::atomic<int>   pending_level_{0};
    std::atomic<int64_t> busy_since_ms_{0};  // ms since epoch; 0 = idle

    static constexpr int64_t TIMEOUT_MS = 3000;
};
```

8.31 src/libs/Motor/swingfeedback.cpp File Reference

```
#include "swingfeedback.h"
#include <atomic>
#include <chrono>
#include <string_view>
#include <sys/timerfd.h>
#include <thread>
#include <unistd.h>
#include <utility>
```

Include dependency graph for swingfeedback.cpp:

Functions

- static int64_t nowMs ()

8.31.1 Function Documentation

8.31.1.1 nowMs()

```
static int64_t nowMs ( )  [static]
```

8.32 src/libs/Piezo/ads1115rpi.cpp File Reference

```
#include "ads1115rpi.h"
#include "i2c_mutex.hpp"
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h>
#include <linux/i2c.h>
#include <string>
#include <stdexcept>
#include <thread>
#include <chrono>
#include <mutex>
```

Include dependency graph for ads1115rpi.cpp:

Functions

- ADS1115settings makeDefaultADS1115Settings ()

8.32.1 Function Documentation

8.32.1.1 makeDefaultADS1115Settings()

```
ADS1115settings makeDefaultADS1115Settings ( )
```

8.33 src/libs/Piezo/event_detector.cpp File Reference

```
#include "event_detector.h"
#include <cstdio>
#include <utility>
```

Include dependency graph for event_detector.cpp:

8.34 src/libs/Piezo/Include/ads1115rpi.h File Reference

```
#include <stdint.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <linux/i2c.h>
#include <linux/i2c-dev.h>
#include <atomic>
#include <thread>
#include <gpiod.hpp>
#include <functional>
#include <memory>
```

Include dependency graph for ads1115rpi.h: This graph shows which files directly or indirectly include this file:

Classes

- struct ADS1115settings
- class ADS1115rpi

Macros

- #define DEBUG

Functions

- ADS1115settings makeDefaultADS1115Settings ()

8.34.1 Macro Definition Documentation

8.34.1.1 DEBUG

```
#define DEBUG
```

8.34.2 Function Documentation

8.34.2.1 makeDefaultADS1115Settings()

```
ADS1115settings makeDefaultADS1115Settings ( )
```

8.35 ads1115rpi.h

Go to the documentation of this file.

```
00001 #ifndef __ADS1115RPI_H
00002 #define __ADS1115RPI_H
00003
00004 #include <stdint.h>
00005 #include <unistd.h>
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <assert.h>
00009 #include <linux/i2c.h>
00010 #include <linux/i2c-dev.h>
00011 #include <atomic>
00012 #include <thread>
00013 #include <gpiod.hpp>
00014 #include <functional>
00015 #include <memory>
00016
00017 #ifndef NDEBUG
00018 #define DEBUG
00019 #endif
00020
00021 struct ADS1115settings
00022 {
00023     int i2c_bus = 1;
00024
00025     static constexpr uint8_t DEFAULT_ADS1115_ADDRESS = 0x48;
00026     uint8_t address = DEFAULT_ADS1115_ADDRESS;
00027
00028     enum SamplingRates
00029     {
00030         FS8HZ = 0,
00031         FS16HZ = 1,
```

```
        FS32HZ = 2,
        FS64HZ = 3,
        FS128HZ = 4,
        FS250HZ = 5,
        FS475HZ = 6,
        FS860HZ = 7
    };

    inline unsigned getSamplingRate() const
    {
        const unsigned SamplingRateEnum2Value[8] =
            {8, 16, 32, 64, 128, 250, 475, 860};
        return SamplingRateEnum2Value[samplingRate];
    }

    SamplingRates samplingRate = FS128HZ;

    enum PGA
    {
        FSR2_048 = 2,
        FSR1_024 = 3,
        FSR0_512 = 4,
        FSR0_256 = 5
    };

    PGA pgaGain = FSR2_048;

    enum Input
    {
        AIN0 = 0,
        AIN1 = 1,
        AIN2 = 2,
        AIN3 = 3
    };

    Input channel = AIN0;

    int drdy_chip = 0;

    static constexpr int DEFAULT_ALERT_RDY_TO_GPIO = 26;
    int drdy_gpio = DEFAULT_ALERT_RDY_TO_GPIO;
};

ADS1115settings makeDefaultADS1115Settings();

class ADS1115rpi
{
public:
    using ADSCallbackInterface = std::function<void(float)>;

    ~ADS1115rpi()
    {
        stop();
    }

    void registerCallback(ADSCallbackInterface ci)
    {
        adsCallbackInterface = ci;
    }

    void setChannel(ADS1115settings::Input channel);

    void start(ADS1115settings settings = ADS1115settings(),
               bool spawn_worker = true);

    ADS1115settings getADS1115settings() const
    {
        return ads1115settings;
    }

    void stop();

    int readOnce();

    float fullScaleVoltage() const
    {
        switch (ads1115settings.pgaGain)
        {
        case ADS1115settings::FSR2_048:
            return 2.048f;
        case ADS1115settings::FSR1_024:
            return 1.024f;
        case ADS1115settings::FSR0_512:
            return 0.512f;
        case ADS1115settings::FSR0_256:
            return 0.256f;
        }
```

```
        assert(false);
        return 0.0f;
    }

    void setRetryEnabled(bool enabled)
    {
        retry_enabled_.store(enabled);
    }

private:
    ADS1115settings ads1115settings;

    void dataReady();
    void worker();

    void i2c_selectDevice();
    void i2c_writeWord(uint8_t reg, unsigned data);
    unsigned i2c_readWord(uint8_t reg);
    int i2c_readConversion();

    const uint8_t reg_config   = 1;
    const uint8_t reg_lo_thres = 2;
    const uint8_t reg_hi_thres = 3;

    std::shared_ptr<gpiod::chip> chip;
    std::shared_ptr<gpiod::line_request> request;
    std::thread thr;
    int fd_i2c = -1;
    bool running = false;
    ADSCallbackInterface adsCallbackInterface;
    std::atomic<bool> retry_enabled_{false};

    static constexpr int64_t ISR_TIMEOUT_MS = 500;
};

#endif
```

8.36 src/libs/Piezo/Include/event_detector.h File Reference

```
#include <functional>
#include "led_controller.h"
```

Include dependency graph for event_detector.h: This graph shows which files directly or indirectly include this file:

Classes

- struct PiezoEventDetectorSettings
- class PiezoEventDetector

8.37 event_detector.h

Go to the documentation of this file.

```
// This header defines the piezo event detector and its settings.
// The detector is responsible for interpreting sensor samples from the ADC,
// tracking press/release state, and exposing higher-level event behaviour
// through a callback while also providing direct LED feedback.


// SOLID principle:
//   S -   The main responsibility of this class is to interpret piezo samples using
//         filtering and threshold logic.
//   O -   New behaviours can be attached through the callback without changing the
//         detector logic itself.
//   L/D - The detector depends on a simpler LED abstraction and a callback rather than
//         on broader application logic, helping to reduce coupling.

#pragma once

#include <functional>

#include "led_controller.h"
```

```

// Stores the threshold and debug settings for the detector.
struct PiezoEventDetectorSettings
{
    float pressThreshold = 0.95f;
    float releaseThreshold = 0.70f;
    bool enableDebugPrints = false;
};

// Declares the detector class and its public interface.
class PiezoEventDetector
{
public:
    // Defines the callback type used for press state changes.
    using ForceCallback = std::function<void(bool)>;

    // Creates the detector with an LED reference and settings.
    PiezoEventDetector(SimpleLEDController& ledRef,
                       PiezoEventDetectorSettings settings);

    void setForceCallback(ForceCallback cb);     // Sets the callback function for state changes.
    void processSample(float v);     // Processes a new sensor sample.
    void reset();    // Resets the detector state.

private:
    SimpleLEDController& led_;    // Reference to the LED controller.
    PiezoEventDetectorSettings settings_;    // Stores the detector settings.
    ForceCallback forceCallback_;    // Stores the callback function.

    // Tracks whether the detector is currently pressed, whether the filter has been initialised and stores the current filtered sample value.
    bool  pressed_     = false;
    bool  filter_init_ = false;
    float filtered_v_  = 0.0f;
};
```

8.38 src/libs/Piezo/ReadMe.md File Reference

8.39 src/main.cpp File Reference

```
#include "positiondetector.hpp"
#include "imureader.hpp"
#include "SwingProcessor.hpp"
#include "SwingDetector.hpp"
#include "swingfeedback.h"
#include "rpi_pwm.h"
#include <cstdlib>
#include <csignal>
#include <unistd.h>
#include <atomic>
#include <thread>
#include <chrono>
#include "ads1115rpi.h"
#include "event_detector.h"
#include "led_controller.h"
#include "ledcallback.hpp"
```

Include dependency graph for main.cpp:

Functions

- static std::atomic< bool > running (true)
- void handle_signal (int)
- int main ()

8.39.1 Function Documentation

8.39.1.1 handle_signal()

```
void handle_signal (
            int  )
```

8.39.1.2 main()

```
int main ( )
```

8.39.1.3 running()

```
static std::atomic< bool > running (
            true  )  [static]
```

Index

_accel_scale_factor
icm20948::ICM20948_I2C, 30
_chip_i2c_master_reset
icm20948::ICM20948_I2C, 27
_current_bank
icm20948::ICM20948_I2C, 30
_enable_data_ready_interrupt
icm20948::ICM20948_I2C, 27
_gyro_scale_factor
icm20948::ICM20948_I2C, 30
_i2c_address
icm20948::ICM20948_I2C, 30
_i2c_bus
icm20948::ICM20948_I2C, 30
_i2c_fd
icm20948::ICM20948_I2C, 30
_magn_scale_factor
icm20948::ICM20948_I2C, 30
_magnetometer_configured
icm20948::ICM20948_I2C, 27
_magnetometer_enable
icm20948::ICM20948_I2C, 27
_magnetometer_init
icm20948::ICM20948_I2C, 27
_magnetometer_set_mode
icm20948::ICM20948_I2C, 27
_magnetometer_set_readout
icm20948::ICM20948_I2C, 27
_read_bit
icm20948::ICM20948_I2C, 27
_read_block_bytes
icm20948::ICM20948_I2C, 27
_read_byte
icm20948::ICM20948_I2C, 27
_read_mag_byte
icm20948::ICM20948_I2C, 28
_set_accel_range_dlpf
icm20948::ICM20948_I2C, 28
_set_accel_sample_rate_div
icm20948::ICM20948_I2C, 28
_set_bank
icm20948::ICM20948_I2C, 28
_set_gyro_range_dlpf
icm20948::ICM20948_I2C, 28
_set_gyro_sample_rate_div
icm20948::ICM20948_I2C, 28
_write_bit
icm20948::ICM20948_I2C, 28
_write_byte
icm20948::ICM20948_I2C, 28
_write_mag_byte
icm20948::ICM20948_I2C, 29
~ADS1115rpi
ADS1115rpi, 17
~IMUReader
IMUReader, 32
~LEDController
LEDController, 36
~PositionDetector
PositionDetector, 44
~RPI_PWM
RPI_PWM, 46
~SimpleLEDController
SimpleLEDController, 51

accel
icm20948::ICM20948_I2C, 30
icm20948::settings, 48
ACCEL_16G
icm20948, 13
ACCEL_2G
icm20948, 13
ACCEL_4G
icm20948, 13
ACCEL_8G
icm20948, 13
ACCEL_DEADBAND
SwingProcessor.hpp, 85
accel_deadband_
SwingProcessor, 64
ACCEL_DLPF_111_4HZ
icm20948, 12
ACCEL_DLPF_11_5HZ
icm20948, 12
ACCEL_DLPF_23_9HZ
icm20948, 12
ACCEL_DLPF_246HZ
icm20948, 12
ACCEL_DLPF_246HZ_2
icm20948, 12
ACCEL_DLPF_473HZ
icm20948, 12
ACCEL_DLPF_50_4HZ
icm20948, 12
ACCEL_DLPF_5_7HZ
icm20948, 12
accel_dlpf_config
icm20948, 12
accel_dlpf_config_to_str

icm20948, 14
accel_scale
icm20948, 13
accel_scale_factor
icm20948, 14
accel_scale_to_str
icm20948, 14
accel_settings
icm20948, 12
icm20948::accel_settings, 15
accumulate
SwingCalibrator, 54
address
ADS1115settings, 22
ADS1115 Pin connection list:, 1
ADS1115rpi, 16
~ADS1115rpi, 17
ads1115settings, 19
ADSCallbackInterface, 17
adsCallbackInterface, 19
chip, 19
dataReady, 17
fd_i2c, 19
fullScaleVoltage, 17
getADS1115settings, 17
i2c_readConversion, 17
i2c_readWord, 18
i2c_selectDevice, 18
i2c_writeWord, 18
ISR_TIMEOUT_MS, 19
readOnce, 18
reg_config, 19
reg_hi_thres, 19
reg_lo_thres, 19
registerCallback, 18
request, 19
retry_enabled_, 20
running, 20
setChannel, 18
setRetryEnabled, 18
start, 18
stop, 18
thr, 20
worker, 19
ads1115rpi.cpp
makeDefaultADS1115Settings, 93
ads1115rpi.h
DEBUG, 94
makeDefaultADS1115Settings, 94
ADS1115settings, 20
address, 22
AIN0, 21
AIN1, 21
AIN2, 21
AIN3, 21
channel, 22
DEFAULT_ADS1115_ADDRESS, 22
DEFAULT_ALERT_RDY_TO_GPIO, 22
drdy_chip, 22
drdy_gpio, 22
FS128HZ, 21
FS16HZ, 21
FS250HZ, 21
FS32HZ, 21
FS475HZ, 21
FS64HZ, 21
FS860HZ, 21
FS8HZ, 21
FSR0_256, 21
FSR0_512, 21
FSR1_024, 21
FSR2_048, 21
getSamplingRate, 22
i2c_bus, 22
Input, 21
PGA, 21
pgaGain, 22
samplingRate, 22
SamplingRates, 21
ads1115settings
ADS1115rpi, 19
ADSCallbackInterface
ADS1115rpi, 17
adsCallbackInterface
ADS1115rpi, 19
AIN0
ADS1115settings, 21
AIN1
ADS1115settings, 21
AIN2
ADS1115settings, 21
AIN3
ADS1115settings, 21
AK09916_CNTL2_ADDR
Icm20948driver.hpp, 69
allOff
LEDController, 37
ax
icm20948::IMUSample, 34
ay
icm20948::IMUSample, 34
az
icm20948::IMUSample, 34

bias_x_
SwingCalibrator, 55
bias_y_
SwingCalibrator, 55
bias_z_
SwingCalibrator, 56
biasX
SwingCalibrator, 54
biasY
SwingCalibrator, 55
biasZ
SwingCalibrator, 55
busy_

SwingFeedback, 60
busy_since_ms_
SwingFeedback, 60

CALIB_SAMPLES
SwingProcessor.hpp, 85
calibrator_
SwingProcessor, 64
callback
PositionDetector, 45
callback_
IMUReader, 33
SwingCalibrator, 56
SwingDetector, 58
cfg
PositionDetector, 45
channel
ADS1115settings, 22
check_DRDY_INT
icm20948::ICM20948_I2C, 29
checkTimeout
SwingFeedback, 60
chip
ADS1115rpi, 19
SimpleLEDController, 52
chip_
IMUReader, 33
RPI_PWM, 47
chipNumber
LEDControllerSettings, 39
SimpleLEDController, 52
classify
SwingDetector, 58
confirmed
PositionDetector, 45
COOLDOWN_MS
SwingProcessor, 64
count_
SwingCalibrator, 56

dataReady
ADS1115rpi, 17
DEBUG
ads1115rpi.h, 94
DEFAULT_ADS1115_ADDRESS
ADS1115settings, 22
DEFAULT_ALERT_RDY_TO_GPIO
ADS1115settings, 22
detect
SwingDetector, 58
dlpf_config
icm20948::accel_settings, 15
icm20948::gyro_settings, 25
dlpf_enable
icm20948::accel_settings, 15
icm20948::gyro_settings, 25
DoneCallback
SwingCalibrator, 54
drdy_chip
ADS1115settings, 22
drdy_gpio
ADS1115settings, 22
duty_
RPI_PWM, 47

enableDebugPrints
PiezoEventDetectorSettings, 42
EventCallback
SwingDetector, 57

fd_i2c
ADS1115rpi, 19
feed
SwingCalibrator, 55
filter_init_
PiezoEventDetector, 41
filtered_v_
PiezoEventDetector, 41
flashGreen
LEDController, 37
SimpleLEDController, 51
ForceCallback
PiezoEventDetector, 40
forceCallback_
PiezoEventDetector, 41
forceOff
SwingFeedback, 60
FS128HZ
ADS1115settings, 21
FS16HZ
ADS1115settings, 21
FS250HZ
ADS1115settings, 21
FS32HZ
ADS1115settings, 21
FS475HZ
ADS1115settings, 21
FS64HZ
ADS1115settings, 21
FS860HZ
ADS1115settings, 21
FS8HZ
ADS1115settings, 21
FSR0_256
ADS1115settings, 21
FSR0_512
ADS1115settings, 21
FSR1_024
ADS1115settings, 21
FSR2_048
ADS1115settings, 21
fullScaleVoltage
ADS1115rpi, 17

getADS1115settings
ADS1115rpi, 17
getDutyCycle
RPI_PWM, 46

getIMU
IMUReader, 32
getSamplingRate
ADS1115settings, 22
gpio_fd_
IMUReader, 33
gpiochip_path_
IMUReader, 33
greenActive_
LEDController, 38
greenGpio
LEDControllerSettings, 39
greenOff
LEDController, 37
greenOffTime_
LEDController, 38
greenOn
LEDController, 37
gx
icm20948::IMUSample, 34
gy
icm20948::IMUSample, 34
gyro
icm20948::ICM20948_I2C, 30
icm20948::settings, 48
GYRO_1000DPS
icm20948, 13
GYRO_2000DPS
icm20948, 13
GYRO_250DPS
icm20948, 13
GYRO_500DPS
icm20948, 13
GYRO_DLPF_119_5HZ
icm20948, 13
GYRO_DLPF_11_6HZ
icm20948, 13
GYRO_DLPF_151_8HZ
icm20948, 13
GYRO_DLPF_196_6HZ
icm20948, 13
GYRO_DLPF_23_9HZ
icm20948, 13
GYRO_DLPF_361_4HZ
icm20948, 13
GYRO_DLPF_51_2HZ
icm20948, 13
GYRO_DLPF_5_7HZ
icm20948, 13
gyro_dlpf_config
icm20948, 13
gyro_dlpf_config_to_str
icm20948, 14
gyro_scale
icm20948, 13
gyro_scale_factor
icm20948, 14
gyro_scale_to_str
icm20948, 14
gyro_settings
icm20948, 12
icm20948::gyro_settings, 25
gz
icm20948::IMUSample, 35

handle_signal
main.cpp, 98
headingToleranceDeg
PositionDetector::Config, 23
HIGH
SwingDetector, 58

i2c1_mutex
i2c_mutex.hpp, 66
I2C_BUS
SwingProcessor.hpp, 85
i2c_bus
ADS1115settings, 22
i2c_mutex.hpp
i2c1_mutex, 66
i2c_readConversion
ADS1115rpi, 17
i2c_readWord
ADS1115rpi, 18
i2c_selectDevice
ADS1115rpi, 18
i2c_writeWord
ADS1115rpi, 18
icm20948, 11
ACCEL_16G, 13
ACCEL_2G, 13
ACCEL_4G, 13
ACCEL_8G, 13
ACCEL_DLPF_111_4HZ, 12
ACCEL_DLPF_11_5HZ, 12
ACCEL_DLPF_23_9HZ, 12
ACCEL_DLPF_246HZ, 12
ACCEL_DLPF_246HZ_2, 12
ACCEL_DLPF_473HZ, 12
ACCEL_DLPF_50_4HZ, 12
ACCEL_DLPF_5_7HZ, 12
accel_dlpf_config, 12
accel_dlpf_config_to_str, 14
accel_scale, 13
accel_scale_factor, 14
accel_scale_to_str, 14
accel_settings, 12
GYRO_1000DPS, 13
GYRO_2000DPS, 13
GYRO_250DPS, 13
GYRO_500DPS, 13
GYRO_DLPF_119_5HZ, 13
GYRO_DLPF_11_6HZ, 13
GYRO_DLPF_151_8HZ, 13
GYRO_DLPF_196_6HZ, 13
GYRO_DLPF_23_9HZ, 13
GYRO_DLPF_361_4HZ, 13

GYRO_DLPF_51_2HZ, 13
GYRO_DLPF_5_7HZ, 13
gyro_dlpf_config, 13
gyro_dlpf_config_to_str, 14
gyro_scale, 13
gyro_scale_factor, 14
gyro_scale_to_str, 14
gyro_settings, 12
MAGN_100HZ, 14
MAGN_10HZ, 13
MAGN_20HZ, 14
MAGN_50HZ, 14
magn_mode, 13
magn_mode_to_str, 14
MAGN_SELF_TEST, 14
magn_settings, 12
MAGN_SHUTDOWN, 13
MAGN_SINGLE, 13
settings, 12
icm20948::accel_settings, 15
accel_settings, 15
dlpf_config, 15
dlpf_enable, 15
sample_rate_div, 16
scale, 16
icm20948::gyro_settings, 24
dlpf_config, 25
dlpf_enable, 25
gyro_settings, 25
sample_rate_div, 25
scale, 25
icm20948::ICM20948_I2C, 25
_accel_scale_factor, 30
_chip_i2c_master_reset, 27
_current_bank, 30
_enable_data_ready_interrupt, 27
_gyro_scale_factor, 30
_i2c_address, 30
_i2c_bus, 30
_i2c_fd, 30
_magn_scale_factor, 30
_magnetometer_configured, 27
_magnetometer_enable, 27
_magnetometer_init, 27
_magnetometer_set_mode, 27
_magnetometer_set_readout, 27
_read_bit, 27
_read_block_bytes, 27
_read_byte, 27
_read_mag_byte, 28
_set_accel_range_dlpf, 28
_set_accel_sample_rate_div, 28
_set_bank, 28
_set_gyro_range_dlpf, 28
_set_gyro_sample_rate_div, 28
_write_bit, 28
_write_byte, 28
_write_mag_byte, 29
accel, 30
check_DRDY_INT, 29
gyro, 30
ICM20948_I2C, 26
init, 29
magn, 30
read_accel_gyro, 29
read_magn, 29
read_sample, 29
reset, 29
set_settings, 29
settings, 31
wake, 29
icm20948::IMUSample, 34
ax, 34
ay, 34
az, 34
gx, 34
gy, 34
gz, 35
mx, 35
my, 35
mz, 35
icm20948::magn_settings, 39
magn_settings, 39
mode, 40
icm20948::settings, 48
accel, 48
gyro, 48
magn, 49
settings, 48
ICM20948_ACCEL_CONFIG_1_ADDR
Icm20948driver.hpp, 69
ICM20948_ACCEL_CONFIG_1_BANK
Icm20948driver.hpp, 69
ICM20948_ACCEL_OUT_BANK
Icm20948driver.hpp, 69
ICM20948_ACCEL_SMPLRT_DIV_1_ADDR
Icm20948driver.hpp, 69
ICM20948_ACCEL_SMPLRT_DIV_1_BANK
Icm20948driver.hpp, 69
ICM20948_ACCEL_SMPLRT_DIV_2_ADDR
Icm20948driver.hpp, 69
ICM20948_ACCEL_SMPLRT_DIV_2_BANK
Icm20948driver.hpp, 69
ICM20948_ACCEL_XOUT_H_ADDR
Icm20948driver.hpp, 69
ICM20948_ACCEL_XOUT_L_ADDR
Icm20948driver.hpp, 70
ICM20948_ACCEL_YOUT_H_ADDR
Icm20948driver.hpp, 70
ICM20948_ACCEL_YOUT_L_ADDR
Icm20948driver.hpp, 70
ICM20948_ACCEL_ZOUT_H_ADDR
Icm20948driver.hpp, 70
ICM20948_ACCEL_ZOUT_L_ADDR
Icm20948driver.hpp, 70
ICM20948_BANK0_WHO_AM_I_VALUE

Icm20948driver.hpp, 70
ICM20948_EXT_SLV_SENS_DATA_00_ADDR
Icm20948driver.hpp, 70
ICM20948_EXT_SLV_SENS_DATA_00_BANK
Icm20948driver.hpp, 70
ICM20948_GYRO_CONFIG_1_ADDR
Icm20948driver.hpp, 70
ICM20948_GYRO_CONFIG_1_BANK
Icm20948driver.hpp, 70
ICM20948_GYRO_OUT_BANK
Icm20948driver.hpp, 71
ICM20948_GYRO_SMPLRT_DIV_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_SMPLRT_DIV_BANK
Icm20948driver.hpp, 71
ICM20948_GYRO_XOUT_H_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_XOUT_L_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_YOUT_H_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_YOUT_L_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_ZOUT_H_ADDR
Icm20948driver.hpp, 71
ICM20948_GYRO_ZOUT_L_ADDR
Icm20948driver.hpp, 71
ICM20948_I2C
icm20948::ICM20948_I2C, 26
ICM20948_I2C_ADDR
Icm20948driver.hpp, 71
ICM20948_I2C_MST_CTRL_ADDR
Icm20948driver.hpp, 72
ICM20948_I2C_MST_CTRL_BANK
Icm20948driver.hpp, 72
ICM20948_I2C_MST_STATUS_ADDR
Icm20948driver.hpp, 72
ICM20948_I2C_MST_STATUS_BANK
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_ADDR_ADDR
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_ADDR_BANK
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_CTRL_ADDR
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_CTRL_BANK
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_REG_ADDR
Icm20948driver.hpp, 72
ICM20948_I2C_SLV0_REG_BANK
Icm20948driver.hpp, 72
ICM20948_I2C_SLV4_ADDR_ADDR
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_ADDR_BANK
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_CTRL_ADDR
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_CTRL_BANK
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_DI_ADDR
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_DI_BANK
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_DO_ADDR
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_DO_BANK
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_REG_ADDR
Icm20948driver.hpp, 73
ICM20948_I2C_SLV4_REG_BANK
Icm20948driver.hpp, 73
ICM20948_INT_ENABLE_1_ADDR
Icm20948driver.hpp, 74
ICM20948_INT_ENABLE_1_BANK
Icm20948driver.hpp, 74
ICM20948_INT_PIN_CFG_ADDR
Icm20948driver.hpp, 74
ICM20948_INT_PIN_CFG_BANK
Icm20948driver.hpp, 74
ICM20948_INT_STATUS_1_ADDR
Icm20948driver.hpp, 74
ICM20948_INT_STATUS_1_BANK
Icm20948driver.hpp, 74
ICM20948_INT_STATUS_ADDR
Icm20948driver.hpp, 74
ICM20948_INT_STATUS_BANK
Icm20948driver.hpp, 74
ICM20948_MAGN_I2C_ADDR
Icm20948driver.hpp, 74
ICM20948_PWR_MGMT_1_ADDR
Icm20948driver.hpp, 74
ICM20948_PWR_MGMT_1_BANK
Icm20948driver.hpp, 75
ICM20948_REG_BANK_SEL_ADDR
Icm20948driver.hpp, 75
ICM20948_REG_BANK_SEL_BANK0_VALUE
Icm20948driver.hpp, 75
ICM20948_REG_BANK_SEL_BANK1_VALUE
Icm20948driver.hpp, 75
ICM20948_REG_BANK_SEL_BANK2_VALUE
Icm20948driver.hpp, 75
ICM20948_REG_BANK_SEL_BANK3_VALUE
Icm20948driver.hpp, 75
ICM20948_TEMP_OUT_H_ADDR
Icm20948driver.hpp, 75
ICM20948_TEMP_OUT_L_ADDR
Icm20948driver.hpp, 75
ICM20948_USER_CTRL_ADDR
Icm20948driver.hpp, 75
ICM20948_USER_CTRL_BANK
Icm20948driver.hpp, 75
ICM20948_WHO_AM_I_ADDR
Icm20948driver.hpp, 76
ICM20948_WHO_AM_I_BANK
Icm20948driver.hpp, 76
Icm20948driver.hpp

AK09916_CNTL2_ADDR, 69
ICM20948_ACCEL_CONFIG_1_ADDR, 69
ICM20948_ACCEL_CONFIG_1_BANK, 69
ICM20948_ACCEL_OUT_BANK, 69
ICM20948_ACCEL_SMPLRT_DIV_1_ADDR, 69
ICM20948_ACCEL_SMPLRT_DIV_1_BANK, 69
ICM20948_ACCEL_SMPLRT_DIV_2_ADDR, 69
ICM20948_ACCEL_SMPLRT_DIV_2_BANK, 69
ICM20948_ACCEL_XOUT_H_ADDR, 69
ICM20948_ACCEL_XOUT_L_ADDR, 70
ICM20948_ACCEL_YOUT_H_ADDR, 70
ICM20948_ACCEL_YOUT_L_ADDR, 70
ICM20948_ACCEL_ZOUT_H_ADDR, 70
ICM20948_ACCEL_ZOUT_L_ADDR, 70
ICM20948_BANK0_WHO_AM_I_VALUE, 70
ICM20948_EXT_SLV_SENS_DATA_00_ADDR, 70
ICM20948_EXT_SLV_SENS_DATA_00_BANK, 70
ICM20948_GYRO_CONFIG_1_ADDR, 70
ICM20948_GYRO_CONFIG_1_BANK, 70
ICM20948_GYRO_OUT_BANK, 71
ICM20948_GYRO_SMPLRT_DIV_ADDR, 71
ICM20948_GYRO_SMPLRT_DIV_BANK, 71
ICM20948_GYRO_XOUT_H_ADDR, 71
ICM20948_GYRO_XOUT_L_ADDR, 71
ICM20948_GYRO_YOUT_H_ADDR, 71
ICM20948_GYRO_YOUT_L_ADDR, 71
ICM20948_GYRO_ZOUT_H_ADDR, 71
ICM20948_GYRO_ZOUT_L_ADDR, 71
ICM20948_I2C_ADDR, 71
ICM20948_I2C_MST_CTRL_ADDR, 72
ICM20948_I2C_MST_CTRL_BANK, 72
ICM20948_I2C_MST_STATUS_ADDR, 72
ICM20948_I2C_MST_STATUS_BANK, 72
ICM20948_I2C_SLV0_ADDR_ADDR, 72
ICM20948_I2C_SLV0_ADDR_BANK, 72
ICM20948_I2C_SLV0_CTRL_ADDR, 72
ICM20948_I2C_SLV0_CTRL_BANK, 72
ICM20948_I2C_SLV0_REG_ADDR, 72
ICM20948_I2C_SLV0_REG_BANK, 72
ICM20948_I2C_SLV4_ADDR_ADDR, 73
ICM20948_I2C_SLV4_ADDR_BANK, 73
ICM20948_I2C_SLV4_CTRL_ADDR, 73
ICM20948_I2C_SLV4_CTRL_BANK, 73
ICM20948_I2C_SLV4_DI_ADDR, 73
ICM20948_I2C_SLV4_DI_BANK, 73
ICM20948_I2C_SLV4_DO_ADDR, 73
ICM20948_I2C_SLV4_DO_BANK, 73
ICM20948_I2C_SLV4_REG_ADDR, 73
ICM20948_I2C_SLV4_REG_BANK, 73
ICM20948_INT_ENABLE_1_ADDR, 74
ICM20948_INT_ENABLE_1_BANK, 74
ICM20948_INT_PIN_CFG_ADDR, 74
ICM20948_INT_PIN_CFG_BANK, 74
ICM20948_INT_STATUS_1_ADDR, 74
ICM20948_INT_STATUS_1_BANK, 74
ICM20948_INT_STATUS_ADDR, 74
ICM20948_INT_STATUS_BANK, 74
ICM20948_MAGN_I2C_ADDR, 74
ICM20948_PWR_MGMT_1_ADDR, 74
ICM20948_PWR_MGMT_1_BANK, 75
ICM20948_REG_BANK_SEL_ADDR, 75
ICM20948_REG_BANK_SEL_BANK0_VALUE, 75
ICM20948_REG_BANK_SEL_BANK1_VALUE, 75
ICM20948_REG_BANK_SEL_BANK2_VALUE, 75
ICM20948_REG_BANK_SEL_BANK3_VALUE, 75
ICM20948_TEMP_OUT_H_ADDR, 75
ICM20948_TEMP_OUT_L_ADDR, 75
ICM20948_USER_CTRL_ADDR, 75
ICM20948_USER_CTRL_BANK, 75
ICM20948_WHO_AM_I_ADDR, 76
ICM20948_WHO_AM_I_BANK, 76
imu_
IMUReader, 33
IMUReader, 31
~IMUReader, 32
callback_, 33
chip_, 33
getIMU, 32
gpio_fd_, 33
gpiochip_path_, 33
imu_, 33
IMUReader, 32
init, 32
rdy_line_, 33
request_, 33
running_, 33
SampleCallback, 32
setCallback, 32
start, 32
stop, 32
stop_fd_, 33
worker, 32
worker_thread_, 33
init
icm20948::ICM20948_I2C, 29
IMUReader, 32
INITIAL
SwingCalibrator, 54
initial_samples_
SwingCalibrator, 56
Input
ADS1115settings, 21
isActive
SwingProcessor, 63
isActive_
SwingProcessor, 64
isArmed_
SwingProcessor, 64
ISR_TIMEOUT_MS
ADS1115rpi, 19
isReady
SwingCalibrator, 55

label
SwingDetector, 58
last_level_
SwingDetector, 58

led_
PiezoEventDetector, 41
LEDController, 35
~LEDController, 36
allOff, 37
flashGreen, 37
greenActive_, 38
greenOff, 37
greenOffTime_, 38
greenOn, 37
LEDController, 36
service, 37
set, 37
updateOutputs, 38
LEDControllerSettings, 38
chipNumber, 39
greenGpio, 39
Level
SwingDetector, 57
loop
RPI_PWM, 46
LOW
SwingDetector, 58

mag_callback_
SwingProcessor, 64
magn
icm20948::ICM20948_I2C, 30
icm20948::settings, 49
MAGN_100HZ
icm20948, 14
MAGN_10HZ
icm20948, 13
MAGN_20HZ
icm20948, 14
MAGN_50HZ
icm20948, 14
magn_mode
icm20948, 13
magn_mode_to_str
icm20948, 14
MAGN_SELF_TEST
icm20948, 14
magn_settings
icm20948, 12
icm20948::magn_settings, 39
MAGN_SHUTDOWN
icm20948, 13
MAGN_SINGLE
icm20948, 13
MagnitudeCallback
SwingProcessor, 62
main
main.cpp, 98
main.cpp
handle_signal, 98
main, 98
running, 98
makeDefaultADS1115Settings
ads1115rpi.cpp, 93
ads1115rpi.h, 94
maxG
PositionDetector::Config, 23
MEDIUM
SwingDetector, 58
minDot
PositionDetector::Config, 23
minG
PositionDetector::Config, 23
mode
icm20948::magn_settings, 40
mx
icm20948::IMUSample, 35
my
icm20948::IMUSample, 35
mz
icm20948::IMUSample, 35

NONE
SwingDetector, 58
nowMs
swingfeedback.cpp, 92
nRefX
PositionDetector, 45
nRefY
PositionDetector, 45
nRefZ
PositionDetector, 45

onForceReady
SwingProcessor, 63
onLevel
SwingFeedback, 60
onPositionReady
SwingProcessor, 63
onSample
PositionDetector, 44
SwingProcessor, 63
operator()
SwingProcessor, 63

pending_level_
SwingFeedback, 61
PGA
ADS1115settings, 21
pgaGain
ADS1115settings, 22
PiezoEventDetector, 40
filter_init_, 41
filtered_v_, 41
ForceCallback, 40
forceCallback_, 41
led_, 41
PiezoEventDetector, 41
pressed_, 41
processSample, 41
reset, 41
setForceCallback, 41

settings_, 42
PiezoEventDetectorSettings, 42
enableDebugPrints, 42
pressThreshold, 42
releaseThreshold, 42
pin
SimpleLEDController, 52
pos_callback_
SwingProcessor, 64
PositionDetector, 43
~PositionDetector, 44
callback, 45
cfg, 45
confirmed, 45
nRefX, 45
nRefY, 45
nRefZ, 45
onSample, 44
PositionDetector, 44
resetState, 44
setCallback, 44
stableCount, 45
StateCallback, 43
wrapAngleDeg, 44
PositionDetector::Config, 23
headingToleranceDeg, 23
maxG, 23
minDot, 23
minG, 23
refHeadingDeg, 23
refX, 24
refY, 24
refZ, 24
stabilitySamples, 24
PositionStateCallback
SwingProcessor, 62
pressed_
PiezoEventDetector, 41
pressThreshold
PiezoEventDetectorSettings, 42
processSample
PiezoEventDetector, 41
pwm_
SwingFeedback, 61
PWM_FREQ_HZ
rpi_pwm.h, 89
PWM_GPIO_LINE
rpi_pwm.h, 89

rdy_line_
IMUReader, 33
read_accel_gyro
icm20948::ICM20948_I2C, 29
read_magn
icm20948::ICM20948_I2C, 29
read_sample
icm20948::ICM20948_I2C, 29
readOnce
ADS1115rpi, 18
READY
SwingCalibrator, 54
RECAL
SwingCalibrator, 54
RECAL_SAMPLES
SwingProcessor.hpp, 85
recal_samples_
SwingCalibrator, 56
refHeadingDeg
PositionDetector::Config, 23
refX
PositionDetector::Config, 24
refY
PositionDetector::Config, 24
refZ
PositionDetector::Config, 24
reg_config
ADS1115rpi, 19
reg_hi_thres
ADS1115rpi, 19
reg_lo_thres
ADS1115rpi, 19
registerCallback
ADS1115rpi, 18
releaseThreshold
PiezoEventDetectorSettings, 42
request
ADS1115rpi, 19
SimpleLEDController, 52
request_
IMUReader, 33
RPI_PWM, 47
reset
icm20948::ICM20948_I2C, 29
PiezoEventDetector, 41
SwingDetector, 58
SwingProcessor, 63
reset_cb_
SwingFeedback, 61
reset_time_
SwingProcessor, 64
ResetCallback
SwingFeedback, 60
resetState
PositionDetector, 44
retry_enabled_
ADS1115rpi, 20
RPI_PWM, 45
~RPI_PWM, 46
chip_, 47
duty_, 47
getDutyCycle, 46
loop, 46
request_, 47
RPI_PWM, 46
running_, 47
setDutyCycle, 46
start, 47

stop, 47
thread_, 47
usToTimerspec, 47
waitUs, 47
rpi_pwm.h
PWM_FREQ_HZ, 89
PWM_GPIO_LINE, 89
running
ADS1115rpi, 20
main.cpp, 98
running_
IMUReader, 33
RPI_PWM, 47

sample_rate_div
icm20948::accel_settings, 16
icm20948::gyro_settings, 25
SampleCallback
IMUReader, 32
samplingRate
ADS1115settings, 22
SamplingRates
ADS1115settings, 21
scale
icm20948::accel_settings, 16
icm20948::gyro_settings, 25
service
LEDController, 37
SimpleLEDController, 51
set
LEDController, 37
SimpleLEDController, 52
set_settings
icm20948::ICM20948_I2C, 29
setCallback
IMUReader, 32
PositionDetector, 44
SwingCalibrator, 55
SwingDetector, 58
setChannel
ADS1115rpi, 18
setDutyCycle
RPI_PWM, 46
setForceCallback
PiezoEventDetector, 41
setMagnitudeCallback
SwingProcessor, 64
setPositionCallback
SwingProcessor, 64
setResetCallback
SwingFeedback, 60
setRetryEnabled
ADS1115rpi, 18
settings
icm20948, 12
icm20948::ICM20948_I2C, 31
icm20948::settings, 48
settings_
PiezoEventDetector, 42
SimpleLEDController, 49
~SimpleLEDController, 51
chip, 52
chipNumber, 52
flashGreen, 51
pin, 52
request, 52
service, 51
set, 52
SimpleLEDController, 51
src/libs/IMU/Icm20948driver.cpp, 65
src/libs/IMU/imureader.cpp, 65
src/libs/IMU/Include/i2c_mutex.hpp, 65, 66
src/libs/IMU/Include/Icm20948driver.hpp, 66, 76
src/libs/IMU/Include/imureader.hpp, 80
src/libs/IMU_math/Include/positiondetector.hpp, 82
src/libs/IMU_math/Include/SwingCalibrator.hpp, 83
src/libs/IMU_math/Include/SwingDetector.hpp, 84
src/libs/IMU_math/Include/SwingProcessor.hpp, 84, 85
src/libs/IMU_math/positiondetector.cpp, 86
src/libs/IMU_math/SwingCalibrator.cpp, 86
src/libs/IMU_math/SwingDetector.cpp, 86
src/libs/IMU_math/SwingProcessor.cpp, 87
src/libs/LEDs/Include/led_controller.h, 87
src/libs/LEDs/Include/ledcallback.hpp, 88
src/libs/LEDs/led_controller.cpp, 88
src/libs/LEDs/ledcallback.cpp, 89
src/libs/Motor/Include/rpi_pwm.h, 89, 90
src/libs/Motor/Include/swingfeedback.h, 91
src/libs/Motor/swingfeedback.cpp, 92
src/libs/Piezo/ads1115rpi.cpp, 93
src/libs/Piezo/event_detector.cpp, 93
src/libs/Piezo/Include/ads1115rpi.h, 93, 94
src/libs/Piezo/Include/event_detector.h, 96
src/libs/Piezo/ReadMe.md, 97
src/main.cpp, 97
stabilitySamples
PositionDetector::Config, 24
stableCount
PositionDetector, 45
start
ADS1115rpi, 18
IMUReader, 32
RPI_PWM, 47
State
SwingCalibrator, 54
state_
SwingCalibrator, 56
StateCallback
PositionDetector, 43
stop
ADS1115rpi, 18
IMUReader, 32
RPI_PWM, 47
stop_fd_
IMUReader, 33
sum_x_
SwingCalibrator, 56

sum_y_
SwingCalibrator, 56
sum_z_
SwingCalibrator, 56
SwingCalibrator, 53
accumulate, 54
bias_x_, 55
bias_y_, 55
bias_z_, 56
biasX, 54
biasY, 55
biasZ, 55
callback_, 56
count_, 56
DoneCallback, 54
feed, 55
INITIAL, 54
initial_samples_, 56
isReady, 55
READY, 54
RECAL, 54
recal_samples_, 56
setCallback, 55
State, 54
state_, 56
sum_x_, 56
sum_y_, 56
sum_z_, 56
SwingCalibrator, 54
triggerRecal, 55
SwingDetector, 57
callback_, 58
classify, 58
detect, 58
EventCallback, 57
HIGH, 58
label, 58
last_level_, 58
Level, 57
LOW, 58
MEDIUM, 58
NONE, 58
reset, 58
setCallback, 58
SwingDetector, 58
SwingFeedback, 59
busy_, 60
busy_since_ms_, 60
checkTimeout, 60
forceOff, 60
onLevel, 60
pending_level_, 61
pwm_, 61
reset_cb_, 61
ResetCallback, 60
setResetCallback, 60
SwingFeedback, 60
TIMEOUT_MS, 61
worker, 60
swingfeedback.cpp
nowMs, 92
SwingProcessor, 61
accel_deadband_, 64
calibrator_, 64
COOLDOWN_MS, 64
isActive, 63
isActive_, 64
isArmed_, 64
mag_callback_, 64
MagnitudeCallback, 62
onForceReady, 63
onPositionReady, 63
onSample, 63
operator(), 63
pos_callback_, 64
PositionStateCallback, 62
reset, 63
reset_time_, 64
setMagnitudeCallback, 64
setPositionCallback, 64
SwingProcessor, 63
SwingProcessor.hpp
ACCEL_DEADBAND, 85
CALIB_SAMPLES, 85
I2C_BUS, 85
RECAL_SAMPLES, 85

thr
ADS1115rpi, 20
thread_
RPI_PWM, 47
TIMEOUT_MS
SwingFeedback, 61
triggerRecal
SwingCalibrator, 55

updateOutputs
LEDController, 38
usToTimerspec
RPI_PWM, 47

waitUs
RPI_PWM, 47
wake
icm20948::ICM20948_I2C, 29
worker
ADS1115rpi, 19
IMUReader, 32
SwingFeedback, 60
worker_thread_
IMUReader, 33
wrapAngleDeg
PositionDetector, 44